AIIT
AI Interview Training

코드 읽기 코스

처음부터 순서대로 — 13개 정류장, 정류장마다 코드 한 줄씩 읽기

AI 1차 채용면접 자동화 플랫폼 · Team AIIT · 2026-11-11 코드 기준

> 2026-11-11 코드 기준. 줄 번호는 코드가 바뀌면 조금씩 어긋날 수 있어요. 어긋나면 함수 이름으로 찾으세요 (VS Code `Ctrl+P` → 파일 이름, `Ctrl+Shift+O` → 함수 이름).

이 가이드는 무엇이 다른가요?

- `코드열람가이드_파일과함수.md` 는 **"이 파일은 누구?"** 를 알려 주는 *인물 소개*예요.
- `프로그램명세서.md` 는 함수 하나하나를 찾아보는 *사전*이에요.
- 이 가이드는 **"어디서 시작해서, 다음엔 어디로 가나?"** 를 알려 주는 *여행 코스*예요. 1번 정류장부터 차례로 따라가면, 마지막엔 프로그램 전체 구조가 머릿속에 그려지게 짰어요.

읽는 방법

- 정류장마다 `🧒 쉬운 설명` → `🔧 전문 설명` → `👀 열어 볼 곳` → `📖 코드 한 줄씩 읽기` → `✅ 체크` 순서예요.
- `👀 열어 볼 곳` 의 경로는 모두 `huge/` 폴더 기준이에요. 예: `backend/app/main.py:113` = `huge/backend/app/main.py` 의 113번째 줄.
- 한 정류장의 `✅ 체크` 를 모두 스스로 답할 수 있으면 다음으로 넘어가요. 못 하겠으면 그 파일을 한 번 더 보거나 물어보세요.
- `📖` **코드 한 줄씩 읽기** 에는 실제 코드를 옮겨 놓고 한 줄씩 풀어 놨어요. 코드 상자 왼쪽 숫자는 **실제 파일의 줄 번호**이고, 설명의 `줄 5` 도 그 번호예요. VS Code 에서 같은 파일을 옆에 열고 같이 보세요.
- 처음 한 바퀴는 `📖` 를 건너뛰고 `🧒·👀·✅` 만 따라가도 돼요. 큰 그림이 잡힌 뒤 두 번째 바퀴에 `📖` 를 읽으면 덜 헷갈려요.
- 처음 보는 문법은 정류장 1 앞의 `🧰` **문법 상자** 에 있어요.
- 함수 안을 처음부터 끝까지 다 이해하려고 하지 마세요. **함수 이름 → 첫 줄 설명(따옴표 글) → 무엇을 부르는지** 세 가지만 보면 충분해요.

🗺 출발 전에 — 큰 그림 한 장

이 프로그램을 **식당**이라고 생각해 봐요.

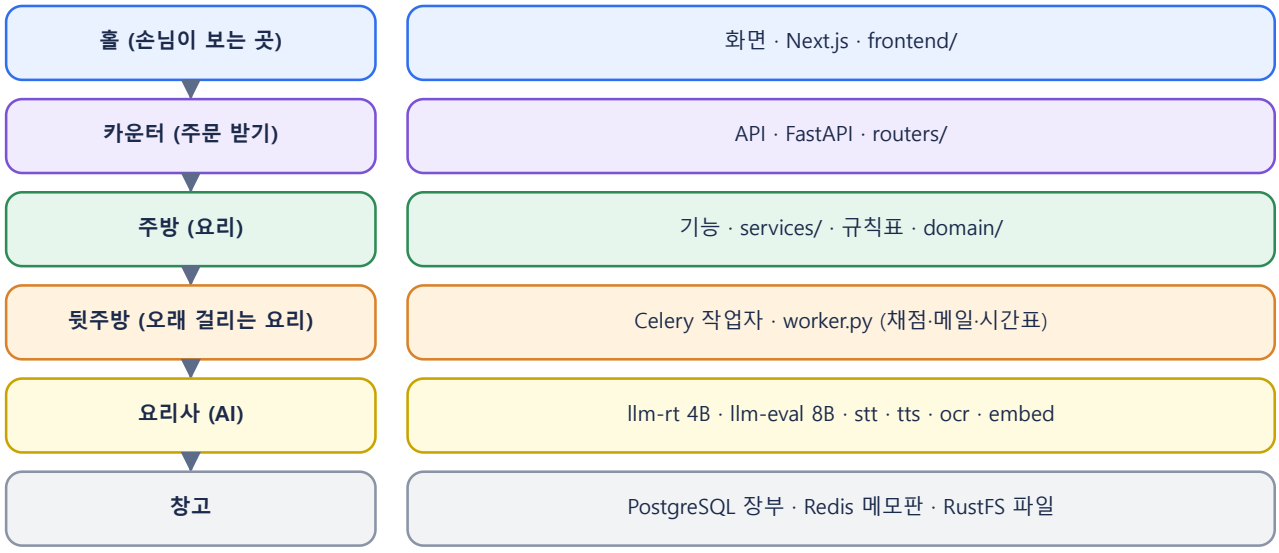


그림. 식당으로 보는 전체 구조

비유	실제	폴더
홀	화면 (Next.js 16, React)	`frontend/`
카운터	API 서버 (FastAPI)	`backend/app/main.py`, `backend/app/routers/`
주방	실제 일을 하는 함수들	`backend/app/services/`, `backend/app/interview/`
규칙판	정해진 규칙(상태, 점수 계산)	`backend/app/domain/`
뒷주방	오래 걸리는 일, 정해진 시각에 하는 일	`backend/app/worker.py`, `backend/app/jobs.py`
창고	DB·파일 저장소·Redis	`compose.infra.yml`
전문 요리사	AI 서버들	`ai/`, `compose.ai.yml`

🔧 **전문 설명:** 브라우저는 Next.js 서버(8080)에만 접속하고, Next.js 가 `/api/*` 를 FastAPI(8000)로 넘겨요(rewrite). 그래서 쿠키와 WebSocket 이 같은 출처에서 동작해요. 무거운 작업(채점, 메일, 코드 실행)은 Celery 워커가 Redis 큐에서 꺼내 처리해요.

코스 전체 순서

정류장	주제	왜 이 순서?
1	전원 켜기	무엇이 몇 개 떠 있는지 알아야 길을 안 잃어요
2	백엔드 출발점 `main.py`	서버가 켜질 때 가장 먼저 실행되는 곳
3	데이터의 모양 `models.py`, `status.py`	모든 함수가 이 데이터를 주고받아요
4	첫 요청 따라가기 — 로그인	화면 → API → DB 의 한 바퀴를 가장 짧게 경험
5	공고 보고 지원하기	같은 한 바퀴를 조금 더 길게
6	면접 질문 만들기	처음 만나는 AI(LLM·RAG)
7	면접 진행	프로그램의 심장. WebSocket·LangGraph
8	코딩테스트	격리된 곳에서 코드 실행
9	채점과 결과	뒷주방(Celery)과 점수 규칙
10	관리자 화면	반대편 손님(관리자)
11	뒤에서 혼자 도는 일	시계 보고 일하는 일꾼들
12	안전장치	권한·오류·암호화
13	테스트로 확인하기	내가 이해한 게 맞는지 직접 돌려 봄



정류장 1~4 를 먼저 · 4번(로그인 한 바퀴)을 이해하면 5번부터는 같은 모양의 반복

그림. 코스 지도 — 13개 정류장

🧰 출발 전 준비물 — 문법 상자



부르기(호출): result = hashPassword("abc123!")

그림. 함수는 자판기



그림. 클래스는 틀, 객체는 그 틀로 만든 하나하나

처음엔 훑어만 보세요. 코스를 가다가 처음 보는 모양이 나오면 여기로 돌아와 찾아보면 돼요.

Python (백엔드)

모양	🧒 쉬운 뜻	🔧 전문 설명
`def 이름(재료):`	자판기 만들기. 재료를 넣으면 무언가 해 줘요	함수 정의
`return 값`	자판기에서 나오는 물건	함수 결과를 돌려주고 끝냄

모양	🧒 쉬운 뜻	🔧 전문 설명
`name: str`	"name 에는 글자가 들어가요" 라는 이름표	타입 힌트. 실행엔 영향 없고 읽기·검사용
`-> dict`	"이 자판기는 사전(dict)을 내놓아요"	반환 타입 힌트
`str | None`	글자이거나, 아무것도 없거나	Union 타입. `None` = 값 없음
`class 이름:`	붕어빵 틀	클래스 정의
`self`	틀로 찍어 낸 "나 자신" 붕어빵	인스턴스 자기 자신
`@무언가` (함수 위)	자판기에 스티커 붙이기. "이 자판기는 /login 주소 담당" 처럼	데코레이터. 함수를 감싸 기능을 덧붙임
`async def` / `await`	"기다리는 동안 다른 손님도 받을 수 있는 자판기" / "여기서 잠깐 기다려요"	비동기 함수 / 결과가 올 때까지 양보하며 대기
`with A as b:`	빌려 쓰고 끝나면 자동으로 반납	컨텍스트 매니저. 끝날 때 정리 코드가 자동 실행
`try: … except X: … finally: …`	해 보고, X 문제가 생기면 이렇게, 마지막엔 무조건 이것	예외 처리
`raise 오류`	"문제 생겼어요!" 하고 손 들기	예외 발생. 위쪽 누군가가 받아 처리
`yield`	"여기서 잠깐 넘겨줄게, 다 쓰면 다시 와"	제너레이터. FastAPI 에선 "빌려주고, 끝나면 정리" 용도
`A or B`	A 가 비어 있으면 B	앞이 거짓 같은 값이면 뒤를 씀
`x if 조건 else y`	조건이 맞으면 x, 아니면 y	조건 표현식
`[x for x in 목록 if 조건]`	목록에서 조건 맞는 것만 골라 새 목록	리스트 컴프리헨션
`Depends(함수)`	"이 창구에 오기 전에 이 문지기를 먼저 거쳐요"	FastAPI 의존성 주입
`db.query(User).filter(...).first()`	사용자 서랍에서 조건 맞는 첫 장 꺼내기	SQLAlchemy 조회 → SQL `SELECT … WHERE … LIMIT 1`
`db.commit()`	"지금까지 고친 것 확정!" 도장	트랜잭션 커밋. 이 전엔 DB 에 진짜로 안 남음

TypeScript / React (프론트엔드)

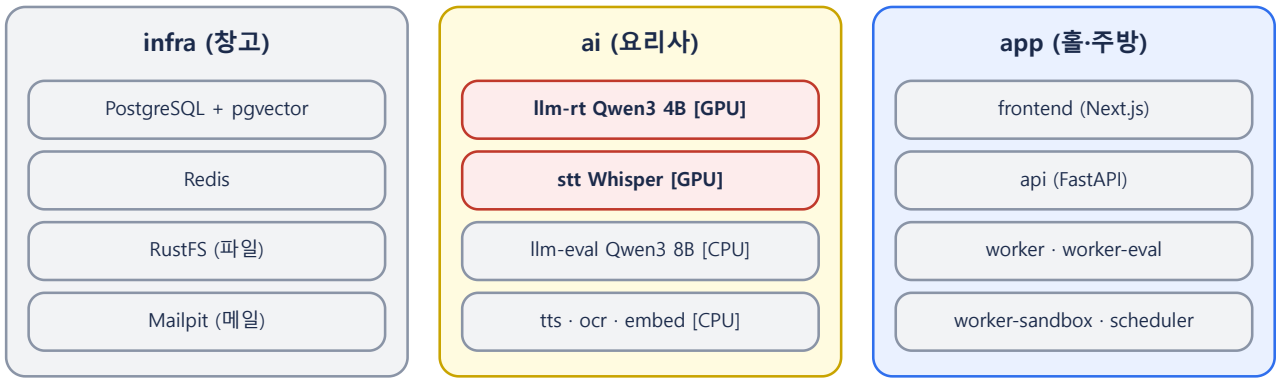
모양	🧒 쉬운 뜻	🔧 전문 설명
`function 이름(재료) { }`	자판기	함수
`const x = …`	이름표 붙인 상자(바꾸지 않음)	재할당 불가 변수
`async` / `await`	Python 과 같아요	Promise 기반 비동기
`api<LoginResponse>(…)`	"돌아오는 건 LoginResponse 모양이에요"	제네릭 타입 인자
`const [값, 바꾸기] = useState(처음값)`	화면이 기억하는 메모. `바꾸기` 를 부르면 화면이 다시 그려져요	React 상태 훅
`useEffect(() => { … }, [a])`	"a 가 바뀔 때마다 이 일을 해요"	부수 효과 훅

모양	🧒 쉬운 뜻	🔧 전문 설명
`useCallback(fn, [a])`	같은 자판기를 계속 재사용	함수 메모이제이션
`<div>…</div>` (JS 안에)	화면 모양 그리기	JSX
`a ?? b`	a 가 없으면(null/undefined) b	Nullish 병합
`` `글자 ${값}` ``	글자 사이에 값 끼워 넣기	템플릿 문자열

정류장 1. 전원 켜기 — 무엇이 떠 있나

🧒 **쉬운 설명:** 식당 문을 열려면 창고, 요리사, 카운터를 순서대로 출근시켜야 해요. `up.sh` 가 "창고 → 요리사 → 카운터·홀" 순서로 출근 도장을 찍어 주는 출근부예요.

🔧 **전문 설명:** Docker Compose 파일이 세 묶음으로 나뉘어 있고, `up.sh` 가 공유 네트워크(`huge-front` , `huge-internal`)를 만든 뒤 `infra → ai → app` 순으로 `docker compose up -d --build` 해요. `huge-internal` 은 `--internal` 이라 바깥 인터넷으로 못 나가요.



그래픽카드 6GB 에는 빨간 두 개만 (실측 5.4 / 6 GB) · 코드만 고치면 app 묶음만 다시 빌드

그림. 컨테이너 3묶음

👀 열어 볼 곳

1. `scripts/up.sh` — 9줄짜리. 전체를 읽어요.
2. `compose.infra.yml` — 창고: `postgres` (DB), `redis` (빠른 메모장·작업 줄), `objectstore` (파일 보관, RustFS), `mailpit` (가짜 우체국)
3. `compose.ai.yml` — 요리사: `llm-rt` (면접 중 빠른 LLM 4B), `llm-eval` (채점용 8B), `stt` (말→글), `tts` (글→말), `embed` (글→숫자 벡터), `ocr` (사진 속 글자 읽기)
4. `compose.app.yml` — 카운터와 일꾼: `api` , `worker-eval` , `worker` , `worker-sandbox` , `scheduler` , `frontend`
5. `README.md` — 켜는 법과 주소

📖 코드 한 줄씩 읽기 — scripts/up.sh (9줄 전부)

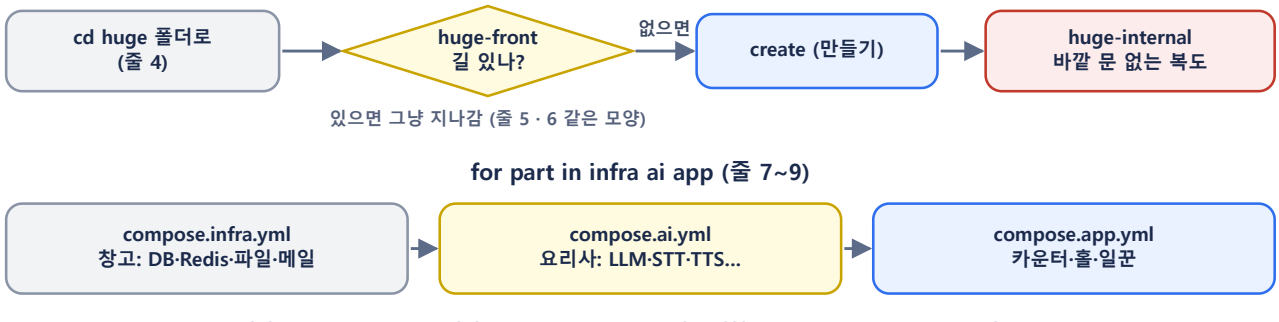


그림. up.sh 가 하는 일

```
1  #!/usr/bin/env sh
2  # 공유 네트워크를 만들고 infra → ai → app 순서로 띄운다.  사용: sh scripts/up.sh [infra|ai|app]
3  set -e
4  cd "$(dirname "$0")/.."
5  docker network inspect huge-front >/dev/null 2>&1 || docker network create huge-front
6  docker network inspect huge-internal >/dev/null 2>&1 || docker network create --internal huge-internal
7  for part in ${1:-infra ai app}; do
8    MSYS_NO_PATHCONV=1 docker compose -f "compose.$part.yml" up -d --build
9  done
```

- **줄 1** `#!/usr/bin/env sh` — 🧑 "이 쪽지는 sh 라는 사람이 읽어요" 라는 표시. 🔧 셔뱅(shebang). 실행할 해석기를 지정해요.
- **줄 2** `#` 로 시작하면 주석. 컴퓨터는 무시하고 사람만 읽어요. 사용법이 적혀 있어요.
- **줄 3** `set -e` — 🧑 "한 줄이라도 실패하면 바로 멈춰!" 창고가 안 열렸는데 카운터를 여는 일을 막아요. 🔧 명령이 0 이 아닌 종료 코드를 내면 스크립트를 즉시 끝내요.
- **줄 4** `cd "$(dirname "$0")/.."` — 안쪽부터 풀어요.
  - `$0` = 지금 실행 중인 이 파일 경로 → `scripts/up.sh`
  - `dirname …` = 그 파일이 든 폴더 → `scripts`
  - `/..` = 한 칸 위 폴더 → `huge`
  - `$( … )` = 안의 명령을 실행한 **결과 글자**로 바꿔 끼움
  - 🧑 어느 폴더에서 이 쪽지를 실행하든 **항상** `huge` **폴더로 먼저 이동**해요. 그래야 다음 줄의 `compose.infra.yml` 을 찾을 수 있어요.
  - 🔧 큰따옴표는 경로에 공백·괄호(`HWF(TeamMAKE)`)가 있어도 한 덩어리로 다루게 해요.
- **줄 5** — 두 부분이 `||` 로 이어져 있어요.
  - `docker network inspect huge-front` = "huge-front 라는 길(네트워크)이 있나 확인"
  - `>/dev/null 2>&1` = 🧑 확인 결과 글자를 화면에 안 띄우고 쓰레기통에 버림. 🔧 표준 출력을 `/dev/null` 로, 표준 오류(2)도 표준 출력(1)과 같은 곳으로 보내요.
  - `||` = "왼쪽이 **실패하면** 오른쪽을 해라". 길이 없으면(확인 실패) → `docker network create huge-front` 로 만들어요.
  - 🧑 "길이 있으면 그냥 지나가고, 없으면 만든다." 두 번 실행해도 안전해요.
  - 🔧 `||` 로 이어진 왼쪽 명령의 실패는 `set -e` 가 멈추지 않아요.
- **줄 6** — 줄 5 와 같은데 `--internal` 이 붙었어요. 🧑 **바깥 인터넷으로 나가는 문이 없는 비밀 복도**예요. DB 와 AI 서버는 이 복도에만 있어서 밖에서 직접 들어올 수 없어요. 🔧 internal 네트워크는 외부 라우팅이 막혀요. 지원자 정보가 밖으로 새지 않게 하는 설계예요.

- **줄 7** `for part in ${1:-infra ai app}; do`
  - `$1` = 이 쪽지를 실행할 때 덧붙인 첫 번째 말. `sh scripts/up.sh app` 이면 `$1` 은 `app`.
  - `${1:-infra ai app}` = "`$1` 이 비어 있으면 `infra ai app` 을 대신 써라"
  - `for part in … ; do … done` = 공백으로 나눈 낱말마다 한 번씩 반복. `part` 에 `infra` → `ai` → `app` 이 차례로 들어가요.
- **줄 8** — 반복할 때마다 하는 일.
  - `MSYS_NO_PATHCONV=1` = 🔧 Windows Git Bash 가 `/data` 같은 경로를 `C:/...` 로 멋대로 바꾸는 걸 막아요. 이 명령 하나에만 적용돼요.
  - `docker compose -f "compose.$part.yml"` = `part` 가 `infra` 면 `compose.infra.yml` 파일을 써요.
  - `up` = 띄우기, `-d` = 뒤에서 조용히 실행(창을 붙잡지 않음), `--build` = 코드가 바뀌었으면 이미지를 새로 만들기.
- **줄 9** `done` = 반복 끝.

🧪 써 보기

명령	결과
`sh scripts/up.sh`	셋 다 순서대로
`sh scripts/up.sh app`	카운터·홀만 (코드 고친 뒤 자주 씀)
`sh scripts/up.sh "ai app"`	요리사와 카운터만 (따옴표로 묶어야 한 덩어리)

✅ 체크

- [ ] 컨테이너가 세 묶음으로 나뉜 이유를 말할 수 있다 (힌트: 따로 끄고 켜기 쉽게)
- [ ] `llm-rt` 와 `llm-eval` 이 왜 둘로 나뉘었는지 짐작할 수 있다 (힌트: 면접 중엔 빨라야 하고, 채점은 정확해야 함)
- [ ] `worker-sandbox` 만 특별한 이유를 찾아봤다 (`compose.app.yml` 에서 `docker.sock` 을 찾아보세요)

정류장 2. 백엔드 출발점 — `main.py`

🧑 **쉬운 설명:** `main.py` 는 카운터의 **개점 준비 체크리스트**예요. 문 열기 전에 창고 정리하고, 어제 하다 만 일을 정리하고, 요리사를 미리 깨워 두고, 각 창구(라우터)를 자리에 배치해요.

🔧 **전문 설명:** FastAPI 앱 객체를 만들고 `lifespan` 으로 시작/종료 작업을 묶어요. 시작 때 테이블 생성 → 칸 추가(migrate) → 기본 데이터 → 저장소 버킷 → 재시작 복구 → LangGraph 체크포인트 연결 → 백그라운드 작업 시작. 라우터 8개를 `include_router` 로 붙여요.

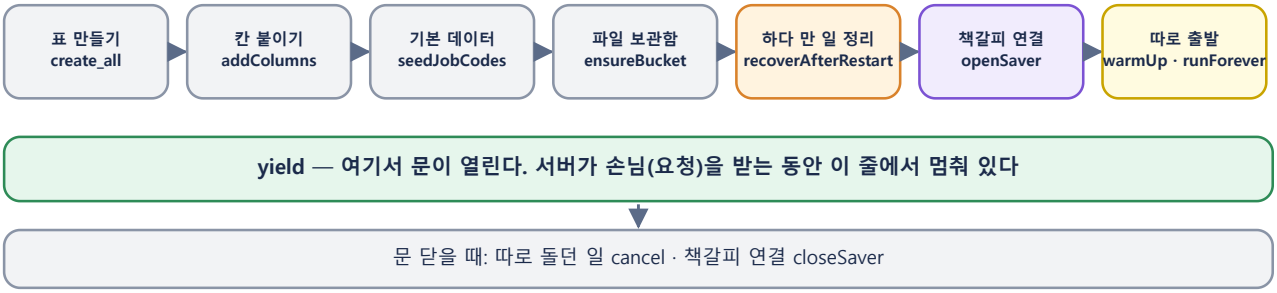


그림. 서버가 켜질 때 (lifespan)

👀 열어 볼 곳 (이 순서로)

순서	위치	무엇을 보나
①	`backend/app/main.py:113`	`app = FastAPI(...)` — 서버 본체가 태어나는 줄
②	`backend/app/main.py:118`	창구 8개(`auth` , `postings` , `applications` , `interview` , `me` , `admin_ops` , `admin_questions` , `admin`)를 붙이는 반복문
③	`backend/app/main.py:69`	`lifespan` — 개점 준비 순서. 한 줄씩 "무슨 일?" 을 말로 바꿔 보세요
④	`backend/app/main.py:43`	`recoverAfterRestart` — 서버가 꺼졌다 켜지면 하다 만 일을 어떻게 정리하나
⑤	`backend/app/main.py:29`	`warmUp` — LLM 을 미리 한 번 불러 깨워 둠 (첫 호출이 45초 걸려서)
⑥	`backend/app/config.py:33`	`loadEnv` — 환경 변수(DB 주소 등)를 읽어 `ENV` 로 만듦
⑦	`backend/app/db.py`	28줄 전부. `engine` (DB 연결), `SessionLocal` (대화 창구), `getDb` (요청마다 창구 하나 빌려주고 돌려받기)

📖 코드 한 줄씩 읽기 — `main.py` , `db.py`

`backend/app/main.py:68` — `lifespan` (개점 준비)

```
68  @asynccontextmanager
69  async def lifespan(_app: FastAPI):
70      Base.metadata.create_all(engine)
71      migrate.addColumns(engine)
72      with SessionLocal() as db:
73          seedJobCodes(db)
74      await asyncio.to_thread(storage.ensureBucket)
75      recoverAfterRestart()
76      await checkpoint.openSaver()  # 면접 진행 그래프 체크포인트 (ADR-07)
77      spawn(warmUp())
78      spawn(jobs.runForever())
79      yield
80      for task in list(backgroundTasks):
81          task.cancel()
82      await checkpoint.closeSaver()
```

- **줄 68** `@asynccontextmanager` — 🧑 "이 함수는 **앞부분(문 열기)** 과 **뒷부분(문 닫기)** 이 있어요" 스티커. 🔧 `yield` 앞은 시작 시, 뒤는 종료 시 실행돼요.
- **줄 69** `_app` 앞의 `_` 는 "받긴 하지만 안 써요" 라는 표시예요.
- **줄 70** `Base.metadata.create_all(engine)` — 🧑 `models.py` 의 서랍 목록표를 보고 **창고에 없는 서랍을 만들어요**. 이미 있으면 그냥 둬요. 🔧 `CREATE TABLE IF NOT EXISTS` . 단, 이미 있는 테이블에 칸을 추가하지는 않아요.
- **줄 71** `migrate.addColumns(engine)` — 그래서 이게 필요해요. 🧑 **이미 있는 서랍에 새로 생긴 칸을 붙여요**. 🔧 `ALTER TABLE … ADD COLUMN` 을 직접 실행하는 간단한 마이그레이션이에요.
- **줄 72~73** `with SessionLocal() as db:` — DB 대화 창구를 하나 빌려서(`db`), `seedJobCodes` 로 **직무 코드 같은 기본 데이터**를 넣고, `with` 가 끝나면 창구를 자동으로 반납해요.
- **줄 74** `await asyncio.to_thread(storage.ensureBucket)` — 🧑 파일 보관함(버킷)이 있는지 확인하고 없으면 만들어요. 이 일은 "기다리는 동안 아무것도 못 하는" 방식이라, **다른 일꾼(스레드)에게 시켜서** 카운터가 멈추지 않게 해요. 🔧 동기 블로킹 함수를 스레드 풀에서 실행해 이벤트 루프를 막지 않아요.
- **줄 75** `recoverAfterRestart()` — 서버가 꺼지기 직전에 하다 만 일을 정리해요 (아래 상자).

- **줄 76** `checkpoint.openSaver()` — 면접 진행표의 **책갈피 보관함**(LangGraph 체크포인트, Postgres)에 연결해요.
- **줄 77~78** `spawn(...)` — 🧒 "이 일은 **기다리지 말고 따로 시작해 둬**". LLM 깨우기(`warmUp`)와 1분마다 도는 정리 일(`runForever`)을 따로 출발시켜요. 🔧 `spawn` (62줄)은 `create_task` 로 태스크를 만들고 `backgroundTasks` 집합에 넣어 가비지 컬렉션으로 사라지지 않게 해요.
- **줄 79** `yield` — 🧒 **여기서 문이 열려요.** 서버가 손님(요청)을 받는 동안 이 줄에서 멈춰 있어요.
- **줄 80~82** 서버를 끌 때 실행. 따로 돌던 일을 멈추고(`cancel`), 책갈피 보관함 연결을 닫아요.

`recoverAfterRestart` 요점 (`main.py:43`)

줄	하는 일	왜?
47	끊긴 이력서 읽기 → 실패로 표시	반쯤 읽다 멈춘 걸 "읽는 중" 으로 영원히 두지 않으려고
48~51	"질문 만드는 중" 이던 지원 → 처음부터 다시 만들기	만들던 일꾼이 서버와 같이 사라졌으니까
53~54	면접 중이던 지원 → `markInterrupted(…, "unknown")` 중단 처리	지원자 탓이 아니니 **이어가기**를 할 수 있게
55	끝나지 않은 면접 연결 기록 → 끝난 시각 찍기	정리
57	`admission.reconcile` — 동시 면접 자리 수를 DB 에 맞춤	Redis 의 자리 수가 어긋났을 수 있어서

`backend/app/main.py:113` — 서버 본체

```
113  app = FastAPI(title="Huge API (Team AIIT)", lifespan=lifespan, docs_url=None, redoc_url=None,
openapi_url=None)
114  app.add_middleware(UploadSizeGuard)
115  app.add_exception_handler(ApiError, apiErrorHandler)
116  app.add_exception_handler(RequestValidationError, validationErrorHandler)
117  app.add_exception_handler(Exception, unexpectedErrorHandler)
118  for module in (auth, postings, applications, interview, me, admin_ops, admin_questions, admin):
119      app.include_router(module.router)
```

- **줄 113** 카운터를 만들어요. `lifespan=lifespan` 으로 위의 개점 준비를 연결해요. `docs_url=None` 등 세 개는 🧒 **"메뉴 전체 목록 자동 안내판" 을 치워요.** 🔧 FastAPI 가 자동으로 만드는 `/docs` API 문서를 꺼서 외부에 API 목록이 드러나지 않게 해요.
- **줄 114** `add_middleware` — 🧒 **카운터 앞 경비원**. 모든 손님이 지나가요. `UploadSizeGuard` 는 너무 큰 이력서 파일을 받기 **전에** 돌려보내요.
- **줄 115~117** "이런 문제가 생기면 이렇게 대답해" 규칙 3개 (정류장 12 참고).
  - `ApiError` → 우리가 일부러 낸 오류 (예: 401 로그인 필요)
  - `RequestValidationError` → 보낸 내용의 모양이 틀림 (422)
  - `Exception` → 예상 못 한 모든 오류 (500, 속 내용은 숨김)
- **줄 118~119** 창구 파일 8개의 `router` 를 카운터에 붙여요. 🔧 각 파일에 `router = APIRouter()` 가 있고, `@router.post("/auth/login")` 같은 주소가 모여 있어요.

`backend/app/db.py` (28줄 전부)

```
13  def makeEngine(url: str):
14      if url.startswith("sqlite"):
15          return create_engine(url, connect_args={"check_same_thread": False})
16      return create_engine(url, pool_pre_ping=True, pool_size=10, max_overflow=10)
```

```
engine = makeEngine(ENV.databaseUrl)
SessionLocal = sessionmaker(bind=engine, expire_on_commit=False)
def getDb() -> Iterator[Session]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

- **makeEngine** — 🧑 창고로 가는 **전화선 묶음**을 만들어요.
  - 주소가 `sqlite` 로 시작하면 → 테스트용 작은 파일 DB. `check_same_thread: False` 는 여러 일꾼이 같이 쓰게 허락하는 설정.
  - 아니면 → 진짜 Postgres. `pool_size=10` = 전화선 10개를 미리 깔아 둠, `max_overflow=10` = 바쁠 땐 10개 더, `pool_pre_ping=True` = 쓰기 전에 "여보세요?" 확인해서 끊긴 선을 안 씀.
- **engine** — 서버 전체가 함께 쓰는 전화선 묶음 하나.
- **SessionLocal** — 🧑 **대화 창구를 찍어 내는 틀**. `SessionLocal()` 하면 창구 하나가 생겨요. 🔧 `expire_on_commit=False` 는 커밋 뒤에도 객체 값을 다시 읽지 않고 그대로 쓰게 해요.
- **getDb** — 요청 하나마다:
  1. 창구 하나 열고 (`SessionLocal()`)
  2. `yield db` 로 **창구 함수에 빌려줘요**
  3. 창구 함수가 끝나면 → `finally` 에서 **무조건 닫아요** (오류가 났어도) - 🔧 라우터에서 `db: Session = Depends(getDb)` 라고 쓰면 FastAPI 가 이 흐름을 자동으로 해 줘요.

✅ 체크

- [ ] 서버가 켜질 때 일어나는 일을 순서대로 5개 이상 말할 수 있다
- [ ] `getDb` 의 `yield` 가 왜 있는지 설명할 수 있다 (힌트: 빌려준 뒤 `finally` 에서 반드시 닫음)
- [ ] "창구 파일 8개" 이름을 보고 각각 무슨 손님을 받을지 짐작해 봤다

정류장 3. 데이터의 모양 — `models.py` 와 `status.py`

🧑 **쉬운 설명:** `models.py` 는 창고의 **서랍 목록표**예요. "사용자 서랍엔 이메일·비밀번호·역할 칸이 있다" 같은 것. `status.py` 는 **보드게임 규칙**이에요. "준비 칸에서는 면접 칸으로만 갈 수 있다" 처럼 말이 어디로 움직일 수 있는지 정해요.

🔧 **전문 설명:** SQLAlchemy ORM 클래스 하나 = 테이블 하나. 지원 1건의 진행 상황은 `applications.attempt_status` 한 칸이 원장(기준)이에요. 상태를 바꿀 때는 `transition(현재, 목표)` 를 거쳐 허용되지 않은 이동을 막아요.

👀 열어 볼 곳

1. `backend/app/models.py` — 다 읽지 말고 **핵심 서랍 5개만**:
   - `User` (18줄) — 사람. `role` 이 `applicant` (지원자) / `admin` (관리자)
   - `JobPosting` (62줄) — 채용 공고
   - `Application` (124줄) — **"누가 어느 공고에 지원했다" 1건**. 이 프로그램의 주인공 서랍
   - `InterviewQuestion` (244줄), `Answer` (267줄) — 면접 질문과 답변
   - `Result` (397줄) — 채점 결과
2. `backend/app/domain/status.py` — 65줄 전부. 특히 `ALLOWED` (17줄) 표를 그림으로 그려 보세요
3. `backend/tests/test_status.py` — 위 규칙이 맞는지 확인하는 테스트. **테스트는 함수의 사용 설명서**예요

그려 볼 상태 지도 (`ALLOWED` 를 그림으로)

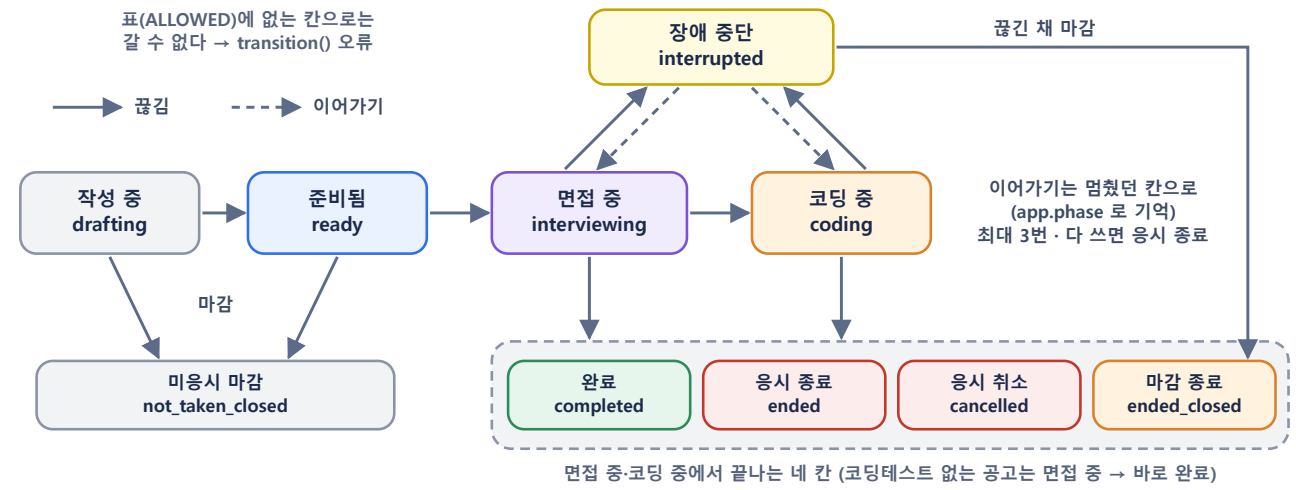


그림. 응시 상태 말판 (status.py ALLOWED)

📖 코드 한 줄씩 읽기 — `models.py`, `status.py`

`backend/app/models.py:18` — 사용자 서랍

```
18  class User(Base):
19      __tablename__ = "users"
20      id: Mapped[str] = mapped_column(String(36), primary_key=True, default=newId)
21      email: Mapped[str] = mapped_column(String(254), unique=True, index=True)
22      password_hash: Mapped[str] = mapped_column(String(255))
23      name: Mapped[str] = mapped_column(String(100))
24      phone: Mapped[str | None] = mapped_column(EncryptedText, nullable=True)
25      role: Mapped[str] = mapped_column(String(16), default="applicant")  # applicant · admin
26      must_change_password: Mapped[bool] = mapped_column(Boolean, default=False)
27      failed_logins: Mapped[int] = mapped_column(Integer, default=0)
28      locked_until: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), nullable=True)
```

- **줄 18** `class User(Base)` — `Base` 를 물려받은 클래스는 **DB 표 하나**가 돼요.
- **줄 19** `__tablename__` — 창고에서 쓰는 진짜 서랍 이름 `users`.
- **줄 20** `id` — 🧑 사람마다 붙는 **주민번호 같은 고유 번호**. `primary_key=True` = 이 칸으로 한 줄을 콕 집어요. `default=newId` = 안 정해 주면 무작위 번호를 자동으로 만들어요. `String(36)` = 36글자(UUID).
- **줄 21** `email` — `unique=True` = 같은 이메일로 두 번 가입 불가(DB 가 막음). `index=True` = 🧑 **책 뒤의 찾아보기**를 만들어 이메일로 빨리 찾아요.
- **줄 22** `password_hash` — 비밀번호 **원본이 아니라 해시**(되돌릴 수 없게 갈아 놓은 값)만 저장해요.
- **줄 24** `phone` — `EncryptedText` = 🧑 **자물쇠 상자에 넣어서** 저장. DB 를 누가 훔쳐봐도 전화번호를 못 읽어요. `nullable=True` = 비워 둬도 돼요.
- **줄 25** `role` — 지원자(`applicant`)인지 관리자(`admin`)인지. 기본은 지원자.
- **줄 26~28** 관리자 초기 비밀번호 변경 여부, 로그인 실패 횟수, 잠금 해제 시각.

🔧 `Mapped[str]` 은 "파이썬에서 이 칸은 str" 이라는 타입 힌트, `mapped_column(...)` 은 "DB 에선 이런 칸" 이라는 설정이에요.

`backend/app/models.py:124` — 지원 서랍 (주인공)

```
124  class Application(Base):
125      __tablename__ = "applications"
127      __table_args__ = (UniqueConstraint("posting_id", "user_id", name="uq_applications_posting_user"),)
128      id: Mapped[str] = mapped_column(String(36), primary_key=True, default=newId)
129      user_id: Mapped[str] = mapped_column(ForeignKey("users.id", ondelete="CASCADE"), index=True)
130      posting_id: Mapped[str] = mapped_column(ForeignKey("job_postings.id"), index=True)
131      attempt_status: Mapped[str] = mapped_column(String(24), default="drafting")
132      analysis_status: Mapped[str] = mapped_column(String(16), default="pending")
133      release_status: Mapped[str] = mapped_column(String(16), default="scheduled")
...
136      resume_snapshot: Mapped[dict | None] = mapped_column(EncryptedJson, nullable=True)
...
140      questions_status: Mapped[str] = mapped_column(String(16), default="none")  # none · composing · ready
· failed
```

- **줄 127** `UniqueConstraint("posting_id", "user_id")` — 🧑 **"한 사람은 한 공고에 한 번만 지원"** 을 창고 자체가 지켜요. 프로그램에 실수가 있어도 DB 가 마지막으로 막아요.
- **줄 129** `ForeignKey("users.id", ondelete="CASCADE")` — 🧑 "이 칸의 값은 **사용자 서랍의 id 중 하나**여야 해요" 라는 연결 끈. `CASCADE` = 사용자가 지워지면 그 사람 지원도 같이 지워져요.
- **줄 131~133** 상태 칸 세 개. 헷갈리지 않게 정리:

칸	무엇의 상태?	예
`attempt_status`	**응시 진행** (원장)	drafting → ready → interviewing → completed
`analysis_status`	**채점 진행**	pending → running → done / needs_review
`release_status`	**결과 공개**	scheduled → released

- **줄 136** `resume_snapshot` — 제출한 이력서 **그 순간의 사진**. 나중에 마이페이지 이력서를 고쳐도 이 지원엔 영향이 없어요. `EncryptedJson` 으로 잠가 둬요.
- **줄 140** `questions_status` — 질문을 다 만들었는지 (none → composing → ready).

`backend/app/domain/status.py:17` — 보드게임 규칙

```
17  ALLOWED = {
18      DRAFTING: {READY, NOT_TAKEN_CLOSED},
19      READY: {INTERVIEWING, NOT_TAKEN_CLOSED},
20      INTERVIEWING: {CODING, COMPLETED, INTERRUPTED, ENDED, CANCELLED, ENDED_CLOSED},
21      CODING: {COMPLETED, INTERRUPTED, ENDED, CANCELLED, ENDED_CLOSED},
22      INTERRUPTED: {INTERVIEWING, CODING, ENDED_CLOSED},
23  }
...
30  def transition(current: str, target: str) -> str:
31      if target not in ALLOWED.get(current, set()):
32          raise TransitionError(f"{current} → {target} 는 허용되지 않는 전이입니다")
33      return target
```

- **줄 17~23** 🧑 "왼쪽 칸에 있는 말은 오른쪽 칸들로만 갈 수 있어요" 라는 표. `{ … }` 안은 **갈 수 있는 칸 목록**이에요.
- **줄 30** `transition(지금, 가고 싶은 곳)`.

- **줄 31** `ALLOWED.get(current, set())` = 표에서 지금 칸의 목록을 찾아요. **표에 없는 칸**(completed 같은 끝 칸)이면 빈 목록 `set()` → 어디로도 못 가요.
  - `target not in …` = 가고 싶은 곳이 목록에 없으면 →
- **줄 32** `raise` 로 손을 들어요. "이 이동은 안 돼요!"
- **줄 33** 괜찮으면 가고 싶은 곳을 그대로 돌려줘요. 그래서 쓰는 쪽은 이렇게 써요: `app.attempt_status = st.transition(app.attempt_status, st.READY)` → 규칙에 맞을 때만 값이 바뀌어요.

✅ **체크**

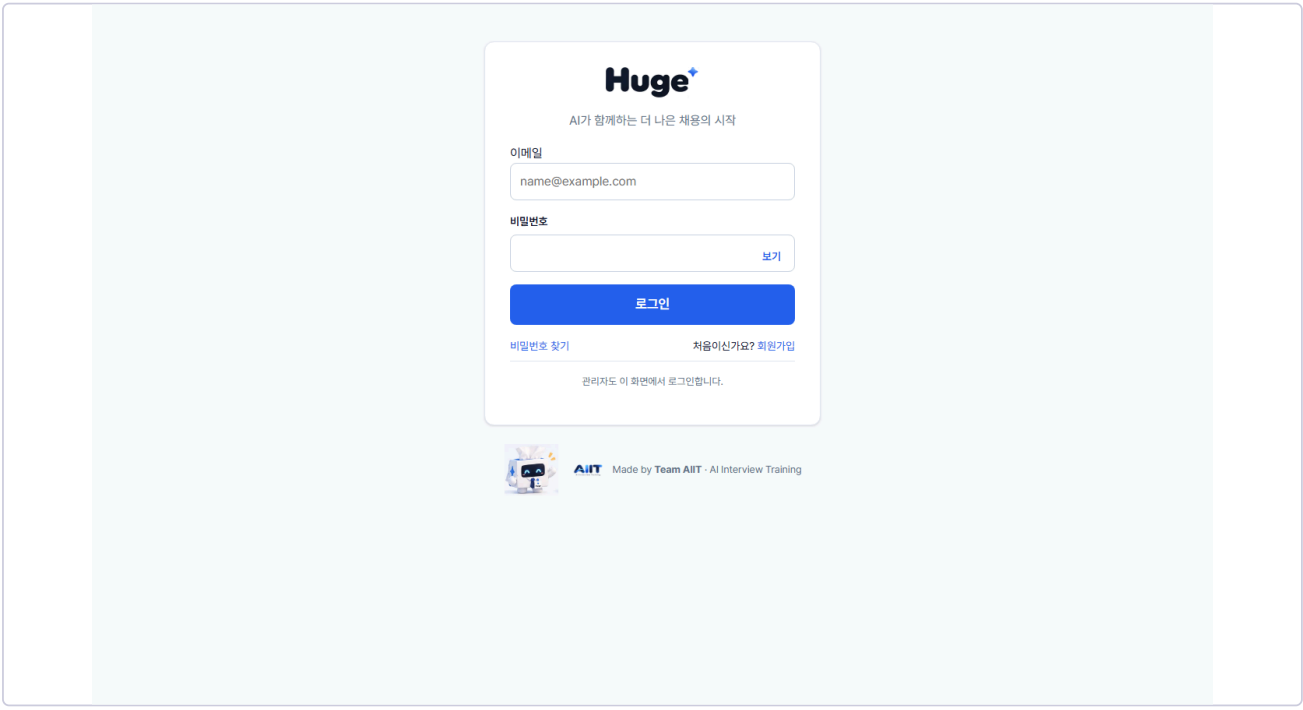
- [ ] `Application` 이 왜 주인공인지 말할 수 있다
- [ ] `ready` 에서 바로 `completed` 로 갈 수 없는 이유를 `ALLOWED` 로 설명할 수 있다
- [ ] `nextAction` (45줄) 이 마이페이지 버튼과 어떤 관계인지 안다

정류장 4. 첫 요청 따라가기 — 로그인 한 바퀴

이제 처음으로 **화면 → 서버 → DB → 화면** 을 한 바퀴 돌아요. 이 바퀴를 한 번 이해하면 다른 기능은 다 같은 모양의 반복이에요.

🧑 **쉬운 설명:** 손님이 "저 철수예요, 비밀번호는 ○○" 라고 말하면 → 홀 직원이 쪽지에 적어 카운터로 → 카운터가 장부와 비교 → 맞으면 **입장 팔찌(쿠키)** 를 채워 줌 → 다음부터는 팔찌만 보여 주면 돼요.

🔧 **전문 설명:** 로그인 성공 시 무작위 토큰을 만들어 해시만 DB(`login_sessions`)에 저장하고, 원본 토큰은 `httponly` 쿠키로 보내요. 이후 요청은 `Depends(currentUser)` 가 쿠키 → 해시 → 세션 행 → 사용자로 바꿔 줘요. 비밀번호는 해시로만 비교해요.



화면. 로그인 (frontend/app/login/page.tsx)

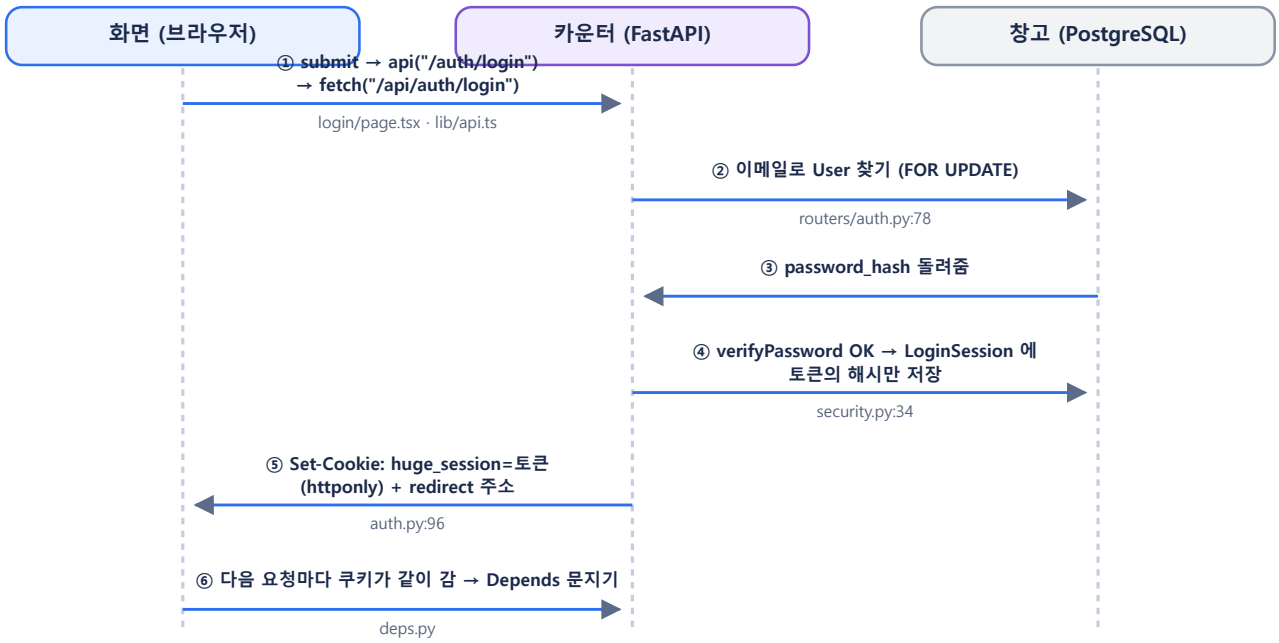


그림. 로그인 한 바퀴 — 화면 → 카운터 → 창고 → 쿠키

👀 따라갈 길 (이 순서 그대로)

순서	위치	무슨 일
①	`frontend/app/layout.tsx:16`	모든 화면을 감싸는 틀. 여기서 `Providers` 를 씌워요
②	`frontend/components/Providers.tsx:23`	"지금 누가 로그인했나(`me`)" 를 모든 화면이 함께 씀
③	`frontend/app/login/page.tsx:31`	`submit` — 버튼 누르면 실행. `api("/auth/login", ...)` 를 불러요
④	`frontend/lib/api.ts:18`	`api` — **모든 화면이 서버에 말 거는 유일한 함수**. 47줄 전부 읽어요
⑤	`frontend/next.config.ts:9`	`rewrites` — `/api/...` 를 FastAPI 로 넘기는 다리
⑥	`backend/app/routers/auth.py:73`	`login` — 카운터에서 받음. 장부(`User`) 찾고, 비밀번호 비교
⑦	`backend/app/security.py:19` , `:23`	`hashPassword` , `verifyPassword` — 비밀번호는 원본을 안 남김
⑧	`backend/app/security.py:34`	`createSession` — 팔찌(토큰) 만들기
⑨	`backend/app/deps.py:12` ~ `:38`	`optionalUser` → `currentUser` → `activeUser` → `requireApplicant` / `requireAdmin` . **문지기 사다리**
⑩	`backend/app/security.py:42`	`resolveSession` — 팔찌를 보고 누군지 알아냄

문지기 사다리 그림

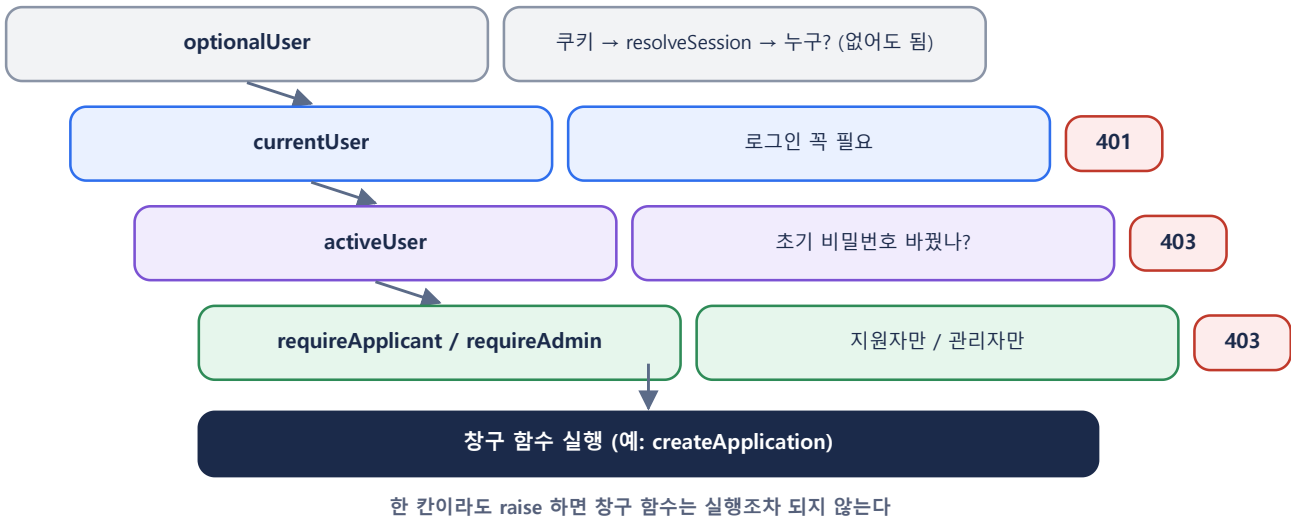


그림. 문지기 사다리 (deps.py)

📖 코드 한 줄씩 읽기 — 화면에서 쿠키까지

① 화면: `frontend/app/login/page.tsx:31` — 버튼을 누르면

```
31      async function submit(event: React.FormEvent) {
32          event.preventDefault();
33          if (!email.trim() || !password) {
34              setError("이메일과 비밀번호를 입력해 주세요.");
35              return;
36          }
37          setBusy(true);
38          setError(null);
39          submitted.current = true;
40          try {
41              const res = await api<LoginResponse>("/auth/login", { body: { email, password, next } });
42              await refreshMe();
43              router.replace(res.redirect);
44          } catch (err) {
45              submitted.current = false;
46              setError((err as ApiError).message);
47          } finally {
48              setBusy(false);
49          }
50      }
```

- **줄 32** `event.preventDefault()` — 🧑 브라우저는 원래 폼을 내면 **페이지를 통째로 새로 고쳐요**. 그걸 막고 우리가 직접 처리해요.
- **줄 33~36** 빈칸이면 서버에 보내지도 않고 바로 안내. `email.trim()` = 앞뒤 공백 제거. `!` = "아니면". `||` = "또는".
- **줄 37** `setBusy(true)` — 버튼을 "처리 중" 으로 (두 번 누르기 방지).
- **줄 39** `submitted.current = true` — "내가 방금 로그인 버튼 눌렀어" 메모. 🔧 `useRef` 라서 바꿔도 화면을 다시 그리지 않아요. 위쪽 `useEffect` 가 "이미 로그인한 채 들어온 사람" 과 구분하는 데 써요.
- **줄 41** ⭐ **서버에 말 거는 줄.** `api("/auth/login", { body: {...} })` → 다음 ②로.

    - `next` = 로그인 전에 가려던 주소 (예: 공고 상세). 로그인 뒤 그리로 돌려보내려고요.
- **줄 42** `refreshMe()` — 🧑 "이제 나 누구야?" 를 서버에 다시 물어서 모든 화면이 아는 `me` 를 새로 채워요 (`Providers.tsx:28`).
- **줄 43** 서버가 알려 준 `redirect` 주소로 이동. 🩹 `replace` 는 뒤로 가기 기록에 로그인 화면을 남기지 않아요.
- **줄 44~46** 실패하면 서버가 준 오류 문장을 그대로 보여 줘요.
- **줄 47~48** 성공이든 실패든 "처리 중" 해제.

② 화면 공용 도구: `frontend/lib/api.ts:18` — 모든 요청이 지나는 문

```
18  export async function api<T = unknown>(path: string, options: { method?: string; body?: Body; signal?:
AbortSignal } = {}): Promise<T> {
19      const { method = options.body === undefined ? "GET" : "POST", body, signal } = options;
20      const init: RequestInit = { method, credentials: "same-origin", signal, cache: "no-store" };
21      if (body instanceof FormData) {
22          init.body = body;
23      } else if (body !== undefined) {
24          init.body = JSON.stringify(body);
25          init.headers = { "content-type": "application/json" };
26      }
27      let res: Response;
28      try {
29          res = await fetch(`/api${path}`, init);
30      } catch (error) {
31          if ((error as Error).name === "AbortError") throw error;
32          throw new ApiError(0, "NETWORK", "서버에 연결하지 못했습니다. 인터넷 연결을 확인해 주세요.");
33      }
34      if (res.status === 204) return undefined as T;
35      const data = await res.json().catch(() => ({}));
36      if (!res.ok) {
38          if (res.status === 401 && !path.startsWith("/auth/") && typeof window !== "undefined")
window.dispatchEvent(new Event(UNAUTHORIZED_EVENT));
39          throw new ApiError(res.status, data.code ?? "UNKNOWN", data.message ?? "요청을 처리하지
못했습니다.");
40      }
41      return data as T;
42  }
```

- **줄 19** 🧑 "보낼 내용(body)이 없으면 **GET**(가져오기), 있으면 **POST**(보내기)". 따로 정해 주면 그걸 써요.
- **줄 20** 편지 봉투 설정.
    - `credentials: "same-origin"` = 🧑 **입장 팔찌(쿠키)를 같이 보내요.** 이게 있어야 서버가 누군지 알아요.
    - `cache: "no-store"` = 옛날 답장을 재사용하지 말고 매번 새로 물어봐요.
- **줄 21~26** 내용물 포장.
    - 파일(`FormData`)이면 그대로.
    - 아니면 `JSON.stringify` 로 **글자로 바꿔** 넣고, "이건 JSON 이에요" 라고 봉투에 적어요.
- **줄 29** ⭐ `fetch(`/api${path}`)` — 진짜로 보내는 줄. `/auth/login` → `/api/auth/login`. 이 `/api` 를 Next.js 가 FastAPI 로 넘겨줘요 (`next.config.ts:9`).
- **줄 30~32** 아예 연결이 안 되면 (인터넷 끊김 등) "서버에 연결하지 못했습니다" 오류로 바꿔요. `AbortError` (사용자가 취소함)는 그대로 넘겨요.
- **줄 34** 204 = "잘 됐고 돌려줄 내용 없음" → 빈 값.
- **줄 35** 답장을 JSON 으로 읽어요. 읽기 실패하면 빈 `{}`.
- **줄 36~39** 실패 답장이면:

    - 401(로그인 끝남)이면 화면 전체에 "로그인 풀렸어요!" 방송 (`dispatchEvent`). `Providers` 가 듣고 `me` 를 지워요.
    - 서버가 준 `{code, message}` 로 `ApiError` 를 만들어 던져요 → ①의 `catch` 가 받아요.
- **줄 41** 성공이면 답장 내용을 돌려줘요.

🩹 **이 함수 하나에 모든 요청을 모은 장점:** 쿠키 보내기, JSON 포장, 오류 모양 통일, 401 처리를 **한 곳에서만** 하면 돼요. 화면 파일 어디에도 `fetch` 가 따로 없어요.

③ 서버 창구: `backend/app/routers/auth.py:73` — 로그인 받기

```
 73  @router.post("/auth/login")
 74  def login(body: LoginRequest, request: Request, response: Response, db: Session = Depends(getDb)):
 75      ipKey = login_block.guardKey(request)
 76      block = login_block.activeBlock(db, ipKey)
 78      user = db.query(User).filter(User.email == body.email.strip().lower()).with_for_update().first()
 79      isAdmin = user is not None and user.role == "admin"
 80      adminLocked = isAdmin and user.locked_until is not None and asKst(user.locked_until) > now()
 81      valid = verifyPassword(user.password_hash if user else DUMMY_HASH, body.password) and user is not None
 82      if adminLocked or not valid:
 83          if isAdmin and not adminLocked:
 84              countAdminFailure(db, user)
 85          if ipKey:
 86              block = login_block.recordFailure(db, ipKey)
 87          failure = login_block.blockedError(db, block) if block else ApiError(401, "INVALID_CREDENTIALS",
INVALID_MESSAGE)
 88          db.commit()
 89          raise failure
 90      if block and not isAdmin:
 91          raise login_block.blockedError(db, block)
 92      user.failed_logins = 0
 93      user.locked_until = None
 94      db.commit()
 95      token = createSession(db, user)
 96      response.set_cookie(COOKIE_NAME, token, httponly=True, samesite="lax", secure=COOKIE_SECURE, path="/")
 97      if user.role == "admin":
 98          redirect = "/password/change" if user.must_change_password else "/admin/postings"
 99      else:
100          redirect = safeNext(body.next) or "/jobs"
101      return {"user_id": user.id, "role": user.role, "password_change_required": user.must_change_password,
"redirect": redirect}
```

- **줄 73** `@router.post("/auth/login")` — 🧑 스티커: "**POST /auth/login** 손님은 이 자판기로". 화면의 `api("/auth/login", {body})` 가 여기로 와요.
- **줄 74** 재료 4개.
    - `body: LoginRequest` = 화면이 보낸 `{email, password, next}`. 🩹 `schemas.py` 의 Pydantic 모델이라 **모양이 틀리면 이 함수에 오기도 전에 422** 로 돌려보내요.
    - `request` = 손님 정보(IP 등), `response` = 돌려보낼 답장(쿠키를 붙이려고)
    - `db = Depends(getDb)` = 창고 창구를 빌려요 (정류장 2).
- **줄 75~76** 🧑 "이 손님 IP 가 요즘 계속 틀려서 **막힌 상태**인가?" 확인. (막는 자세한 규칙은 `services/login_block.py`)
- **줄 78** 이메일로 사용자를 찾아요.
    - `.strip().lower()` = 앞뒤 공백 빼고 소문자로. `Kim@A.com` 과 `kim@a.com` 을 같은 사람으로.

    - `.with_for_update()` = 🧑 **"내가 이 장부 줄을 보는 동안 다른 사람은 기다려!"** 동시에 수십 번 시도해서 실패 횟수 세기를 피해 가는 걸 막아요. 🪛 `SELECT … FOR UPDATE` 행 잠금.
    - `.first()` = 첫 번째 하나, 없으면 `None`.
- **줄 79~80** 관리자인지, 관리자 계정이 잠겨 있는지.
- **줄 81** ⭐ 비밀번호 확인. 꼼꼼히 보세요:
    - `user.password_hash if user else DUMMY_HASH` = 사람이 있으면 그 해시, **없으면 가짜 해시**와 비교해요.
    - 🧑 왜 가짜와 비교할까요? 없는 이메일이면 바로 "틀림" 이라고 하면 **대답이 빨라요**. 나쁜 사람이 대답 속도로 "이 이메일은 가입돼 있구나" 를 알아챌 수 있어요. 그래서 없는 사람이어도 **똑같이 시간이 걸리게** 해요. 🪛 타이밍 공격 방지.
    - `and user is not None` = 가짜 해시가 우연히 맞아도 사람이 없으면 실패.
- **줄 82~89** 실패했을 때.
    - 관리자면 실패 횟수 +1 (`countAdminFailure` , 여러 번이면 계정 잠금)
    - IP 실패 기록 +1 (`recordFailure`)
    - 오류 문장은 **"이메일 또는 비밀번호가 올바르지 않습니다"** 하나. 🧑 "이메일이 틀렸어요" 라고 따로 말하면 가입 여부를 알려 주는 셈이라 일부러 뭉뚱그려요.
    - `db.commit()` 으로 실패 기록을 **먼저 확정**한 뒤 `raise` . (raise 하면 함수가 끝나니까 순서가 중요해요)
- **줄 90~91** 비밀번호는 맞았지만 IP 가 막혀 있으면 그래도 거절 (관리자는 예외 — 관리자가 같은 IP 공격 때문에 못 들어오는 걸 막으려고).
- **줄 92~94** 성공! 실패 횟수와 잠금을 0 으로.
- **줄 95** `createSession` — 입장 팔찌 만들기 (④).
- **줄 96** ⭐ 팔찌를 쿠키로 채워 줘요.
    - `httponly=True` = 🧑 **화면의 자바스크립트가 팔찌를 만질 수 없어요.** 나쁜 스크립트가 끼어들어도 팔찌를 훔쳐 가지 못해요.
    - `samesite="lax"` = 다른 사이트에서 몰래 보낸 요청에는 팔찌를 안 붙여요.
    - `secure` = https 일 때만 보내기 (주소가 https 로 시작할 때 켜짐).
- **줄 97~100** 어디로 보낼지. 관리자 → 초기 비밀번호면 변경 화면, 아니면 공고 관리. 지원자 → `safeNext` 로 **안전한 우리 주소일 때만** 가려던 곳, 아니면 공고 목록.
- **줄 101** 화면에 돌려주는 답장. ①의 `res.redirect` 가 이 값이에요.

④ 팔찌 만들기·읽기: `backend/app/security.py`

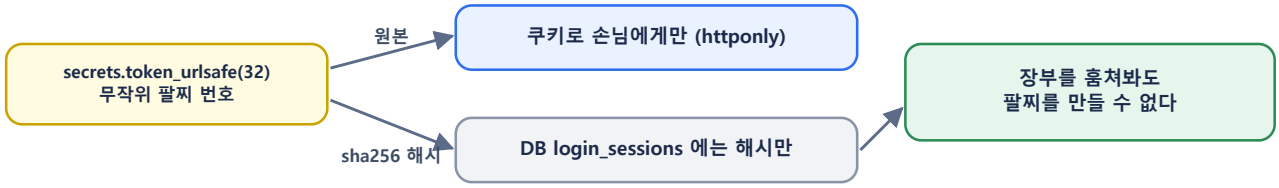


그림. 원본은 손님에게, 해시는 장부에

```
30  def tokenHash(token: str) -> str:
31      return hashlib.sha256(token.encode()).hexdigest()
34  def createSession(db: Session, user: User) -> str:
35      token = secrets.token_urlsafe(32)
36      hours = getSetting(db, "session_idle_hours")
37      db.add(LoginSession(token_hash=tokenHash(token), user_id=user.id, expires_at=now() +
timedelta(hours=hours)))
38      db.commit()
39      return token
```

```
42  def resolveSession(db: Session, token: str | None, touch: bool = True) -> User | None:
43      if not token:
44          return None
45      row = db.get(LoginSession, tokenHash(token))
46      if row is None or asKst(row.expires_at) <= now():
47          return None
48      if touch:
49          row.expires_at = now() + timedelta(hours=getSetting(db, "session_idle_hours"))
50          db.commit()
51      return db.get(User, row.user_id)
```

- **줄 35** `secrets.token_urlsafe(32)` — 🧑 **아무도 맞힐 수 없는 긴 무작위 글자**. 이게 팔찌 번호예요.
- **줄 37** ⭐ DB 에는 팔찌 번호 **원본이 아니라 해시**(`tokenHash`)를 적어요. 🧑 창고 장부를 누가 훔쳐봐도 팔찌를 위조할 수 없어요.
  - `expires_at` = 지금 + 설정 시간(기본 2시간) → 그때까지 유효.
- **줄 39** 원본은 손님에게만 (쿠키로) 줘요.
- **줄 43~44** 팔찌가 없으면 "모르는 사람".
- **줄 45** 받은 팔찌를 같은 방법으로 해시해서 장부에서 찾아요.
- **줄 46~47** 없거나 시간이 지났으면 "모르는 사람".
- **줄 48~50** 🧑 **쓸 때마다 2시간 연장.** 계속 쓰는 동안은 안 풀리고, 2시간 동안 아무것도 안 하면 풀려요. 🔧 슬라이딩 만료.
- **줄 51** 그 팔찌 주인(사용자)을 돌려줘요.

⑤ 문지기 사다리: `backend/app/deps.py:12`

```
12  def optionalUser(request: Request, db: Session = Depends(getDb)) -> User | None:
13      return resolveSession(db, request.cookies.get(COOKIE_NAME))
16  def currentUser(user: User | None = Depends(optionalUser)) -> User:
17      if user is None:
18          raise ApiError(401, "LOGIN_REQUIRED", "로그인이 필요합니다.")
19      return user
22  def activeUser(user: User = Depends(currentUser)) -> User:
24      if user.must_change_password:
25          raise ApiError(403, "PASSWORD_CHANGE_REQUIRED", "비밀번호를 먼저 변경해 주세요.")
26      return user
29  def requireApplicant(user: User = Depends(activeUser)) -> User:
30      if user.role != "applicant":
31          raise ApiError(403, "FORBIDDEN", "접근 권한이 없습니다.")
32      return user
```

- 🧑 **문지기가 앞 문지기를 부르는 사다리**예요. 각 함수 괄호 안의 `Depends(앞 문지기)` 를 보세요.
  - `requireApplicant` 는 `activeUser` 를 먼저 부르고 → `activeUser` 는 `currentUser` 를 → `currentUser` 는 `optionalUser` 를 → `optionalUser` 는 쿠키에서 팔찌를 꺼내 `resolveSession` (④) 을 불러요.
  - 아래에서 위로 실행되고, 중간에 하나라도 `raise` 하면 **창구 함수는 실행조차 안 돼요.**
- **줄 13** `request.cookies.get(COOKIE_NAME)` = 손님 쿠키에서 `huge_session` 팔찌 꺼내기.
- 🔧 401 = "누군지 몰라요(로그인 필요)", 403 = "누군지는 아는데 여긴 못 들어와요".

이제 이 줄을 읽을 수 있어요:

```
def createApplication(body: ApplicationCreate, db: Session = Depends(getDb), user: User =
Depends(requireApplicant)):
```

→ "지원 만들기 창구. 창고 창구를 빌리고, **로그인했고·비밀번호 바꿨고·지원자인 사람**만 들어와요. 그 사람이 `user` 예요."

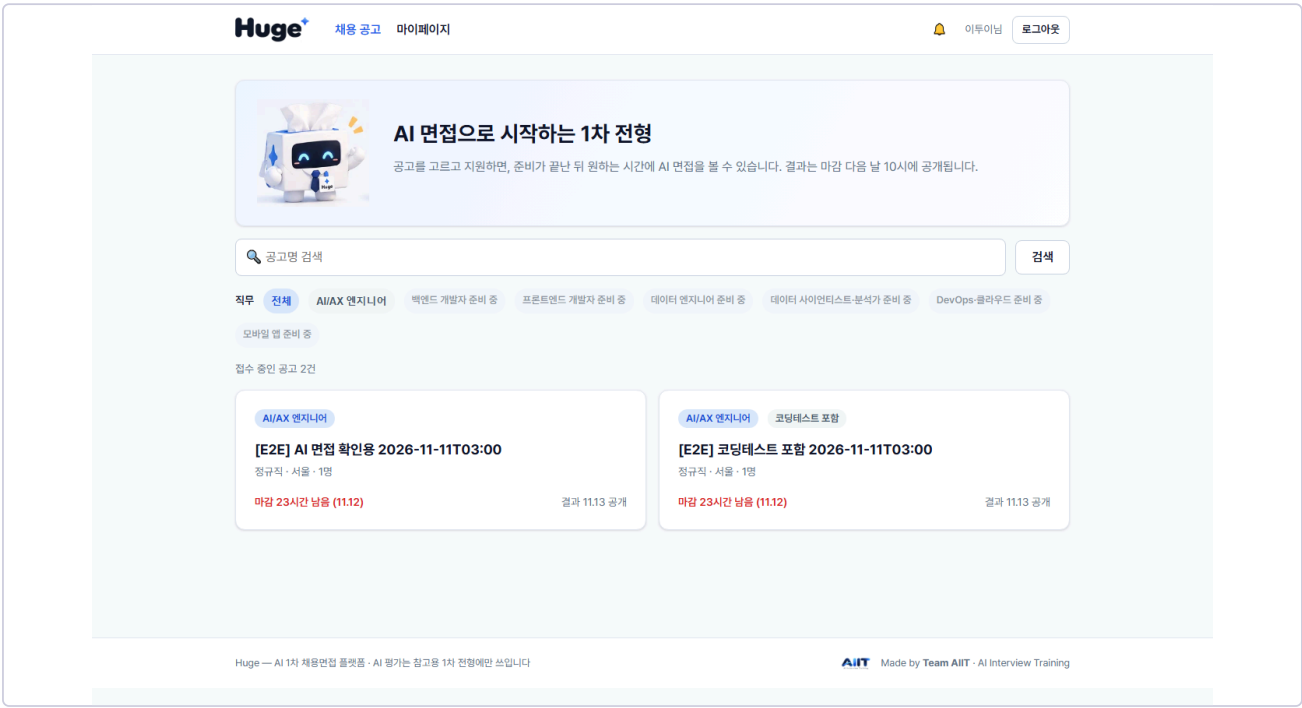
✅ **체크**

- [ ] 로그인 버튼부터 쿠키를 받을 때까지 지나가는 파일을 순서대로 말할 수 있다
- [ ] 화면 코드에 `fetch` 가 여기저기 없고 `api()` 하나만 쓰는 장점을 말할 수 있다
- [ ] 라우터 함수 괄호 안의 `user: User = Depends(requireApplicant)` 를 보면 "지원자만 들어오는 창구" 라고 바로 읽힌다
- [ ] 🧪 직접 해 보기: `backend/tests/test_api_auth.py` 의 `test_signupThenLoginRedirectsToJobs` 를 읽고 `python -m pytest tests/test_api_auth.py -q` 를 `huge/backend` 에서 돌려 본다

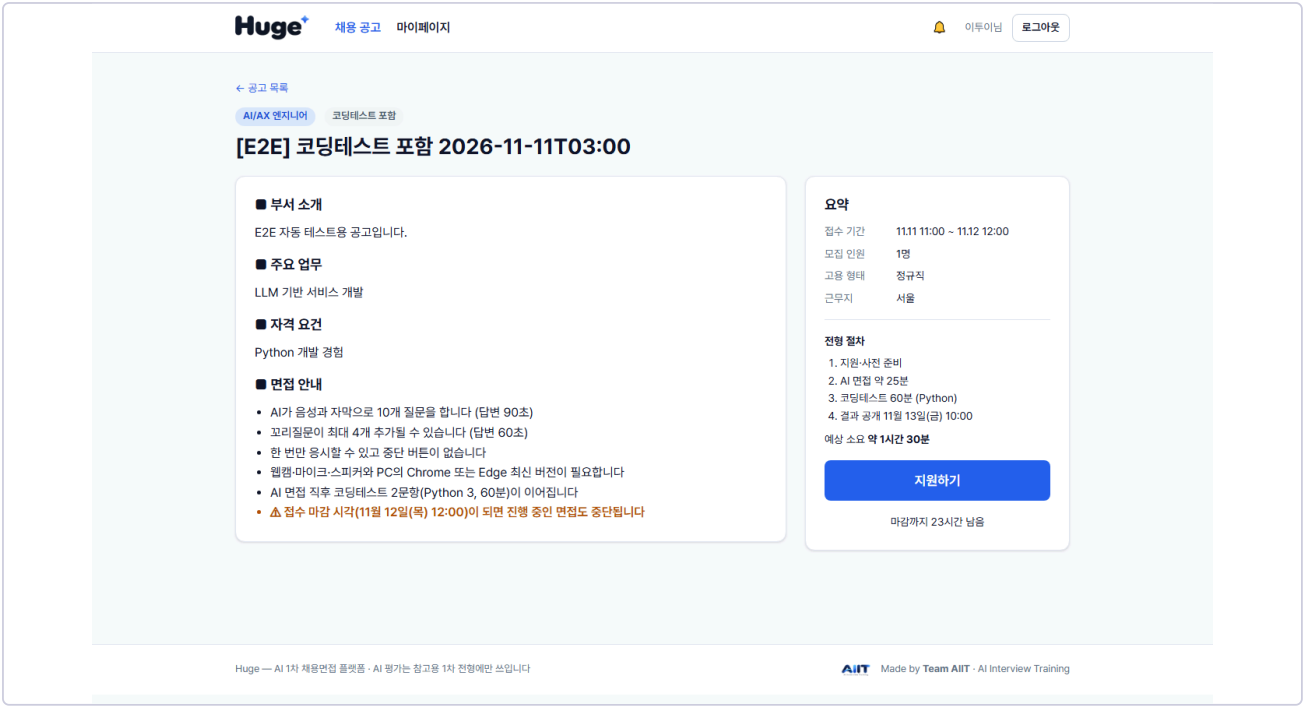
정류장 5. 공고 보고 지원하기 — 같은 바퀴, 조금 더 길게

🧒 **쉬운 설명:** 메뉴판(공고 목록)을 보고 → 메뉴 하나를 골라(지원) → 약관에 사인하고(동의) → 내 소개서를 내면(이력서) → 주방이 **나만을 위한 질문을 몰래 준비하기 시작**해요.

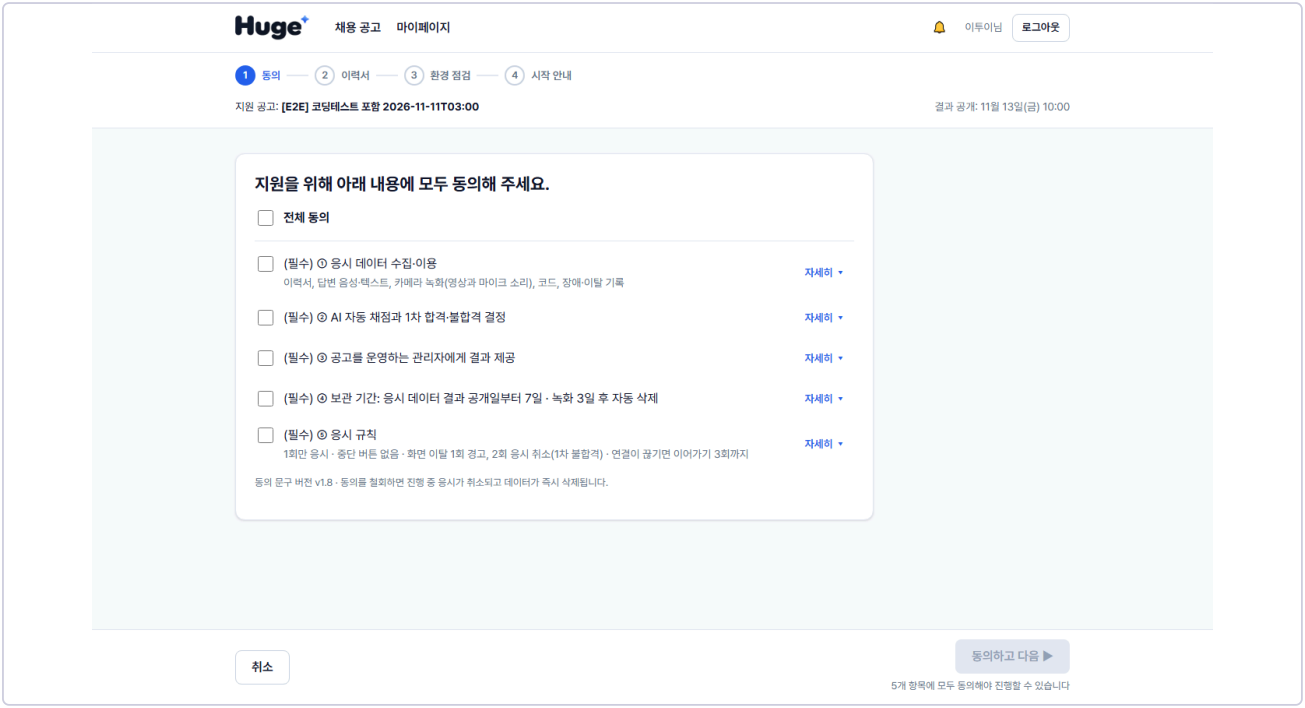
🧪 **전문 설명:** 지원 생성 시 `Application` 이 `drafting` 으로 만들어지고, 이력서 확정(`submitResume`)에서 `ready` 로 전이한 뒤 `question_compose.scheduleCompose` 로 비동기 질문 구성을 시작해요. 지원 단계 화면들은 `ApplyFlow` 가 공통으로 감싸 단계 순서를 지키게 해요.



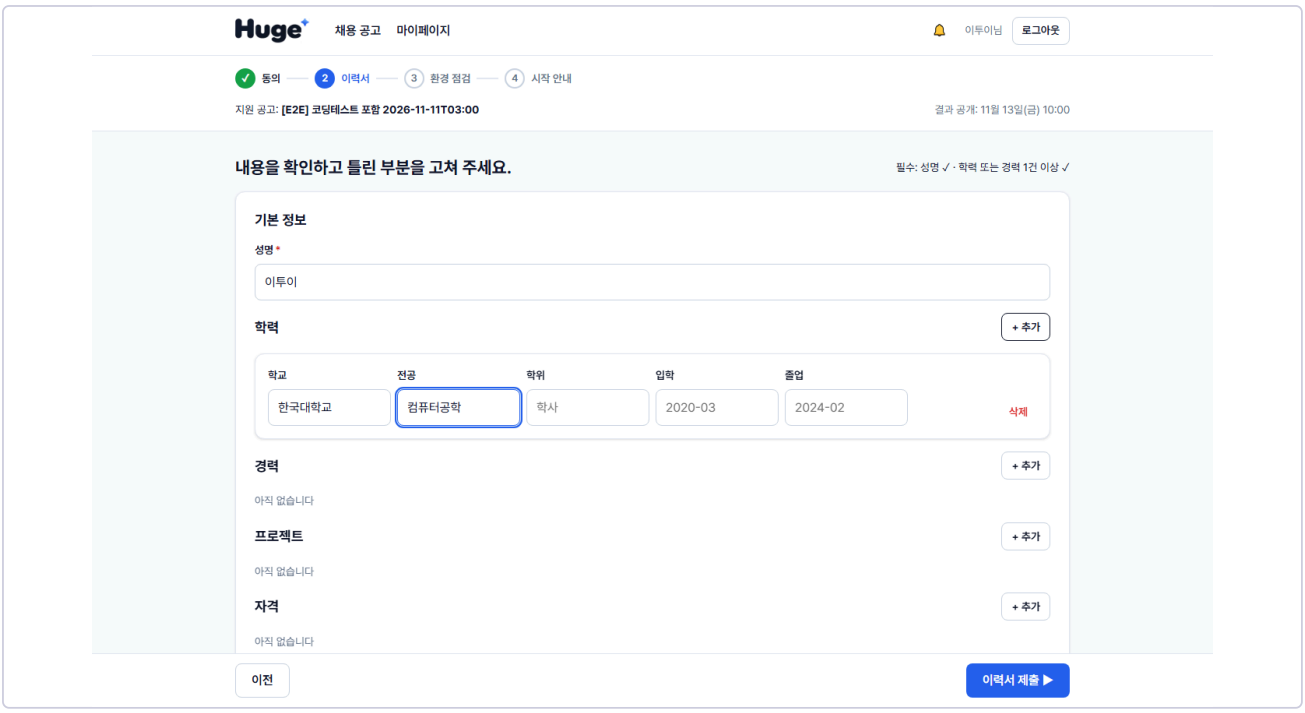
화면. 공고 목록 (app/jobs/page.tsx)



화면. 공고 상세 — [지원하기]가 createApplication 을 부른다



화면. 동의 5개 (apply/.../consent)



화면. 이력서 확인 — [이력서 제출]이 submitResume 을 부른다

👀 따라갈 길

순서	화면(홀)	서버(카운터)
① 공고 목록	`frontend/app/jobs/page.tsx:25` `JobList`	`backend/app/routers/postings.py:23` `listPostings`
② 공고 상세·지원	`frontend/app/jobs/[postingId]/page.tsx:43` `apply`	`backend/app/routers/applications.py:43` `createApplication`
③ 단계 틀	`frontend/components/ApplyFlow.tsx:94` `ApplyFlow` — 동의·이력서·점검·규칙 4단계를 감싸요	`backend/app/routers/applications.py:65` `getApplication`
④ 동의	`frontend/app/apply/[applicationId]/consent/page.tsx:36` `submit`	`backend/app/routers/applications.py:92` `consent`
⑤ 이력서 올리기	`frontend/app/apply/[applicationId]/resume/page.tsx:106` `sendFile`	`backend/app/routers/applications.py:149` `uploadResume` → `services/resume_extract.py:346` `extractUpload` (PDF 글자 읽기 → OCR → LLM 정리)
⑥ 이력서 확정	`frontend/app/apply/[applicationId]/resume/confirm/page.tsx:61`	`backend/app/routers/applications.py:205` `submitResume` ⭐

⭐ `submitResume` **은 꼭 끝까지 읽어요** (약 25줄). 이 함수 하나에 지금까지 배운 게 다 들어 있어요: `ownedApplication`(내 것만) → 규칙 확인 → `st.transition(..., READY)`(정류장 3) → `db.commit()` → `scheduleCompose`(다음 정류장으로 가는 문).

📖 코드 한 줄씩 읽기 — routers/applications.py

`applications.py:43` — 지원 만들기

```
42  @router.post("/applications")
43  def createApplication(body: ApplicationCreate, db: Session = Depends(getDb), user: User =
Depends(requireApplicant)):
44      posting = db.get(JobPosting, body.posting_id)
45      if posting is None or posting.status == "draft":
46          raise notFound()
48      db.query(User).filter(User.id == user.id).with_for_update().one()
49      existing = db.query(Application).filter_by(user_id=user.id, posting_id=posting.id).first()
50      if existing is not None and existing.attempt_status != st.DRAFTING:
51          raise ApiError(409, "ALREADY_APPLIED", "이미 지원한 공고입니다. 진행 상황은 마이페이지에서 확인할 수
있습니다.")
52      if not isAcceptingApplications(posting):
53          raise ApiError(409, "POSTING_CLOSED", "접수 기간이 아닙니다.")
54      if posting.job_code != "AI_AX":
55          raise ApiError(400, "JOB_NOT_OPEN", "준비 중인 직무입니다.")
56      if existing is not None:
57          return {"application_id": existing.id, "attempt_status": existing.attempt_status, "next_step":
nextStep(db, existing)}
58      app = Application(user_id=user.id, posting_id=posting.id, attempt_status=st.DRAFTING)
59      db.add(app)
60      db.commit()
61      return {"application_id": app.id, "attempt_status": app.attempt_status, "next_step": "consent"}
```

- **줄 44** `db.get(JobPosting, 번호)` = 공고 서랍에서 번호로 한 장 꺼내기.
- **줄 45~46** 없거나 아직 임시저장(draft)인 공고면 **404 "찾을 수 없습니다"**. 🧑 임시저장 공고가 "있다" 는 사실조차 알려 주지 않아요.
- **줄 48** 🧑 **"이 사람의 지원 만들기는 한 줄로 서!"** 버튼을 빠르게 두 번 눌러 지원이 2건 생기는 걸 막아요. 결과는 안 쓰고 잠금만 걸어요.
- **줄 49** 이미 이 공고에 지원했는지 찾아요.
- **줄 50~51** 이미 지원했고 **작성 중이 아니면**(제출 끝) → 409 "이미 지원". 🔧 409 = 지금 상태와 충돌.
- **줄 52~55** 접수 기간인지, 열린 직무(지금은 AI 직무 `AI_AX` 만)인지.
- **줄 56~57** 작성 중인 지원이 이미 있으면 **새로 안 만들고 그걸 이어서** 하게 해요. `nextStep` 이 동의를 했으면 `resume`, 안 했으면 `consent` 를 알려줘요.
- **줄 58~60** 새 지원을 `drafting` 으로 만들고 → `db.add` (서랍에 넣을 준비) → `db.commit` (확정).
- **줄 61** 화면에 "다음은 동의 단계" 라고 알려줘요.

`applications.py:205` — 이력서 확정 ⭐

```
204  @router.post("/applications/{applicationId}/resume")
205  async def submitResume(applicationId: str, body: ResumeSubmit, db: Session = Depends(getDb), user: User =
Depends(requireApplicant)):
206      app = ownedApplication(db, user, applicationId, forUpdate=True)
207      requireDrafting(app)
```

```
208     if nextStep(db, app) == "consent":
209         raise ApiError(409, "CONSENT_REQUIRED", "필수 동의를 먼저 해 주세요.")
210     if not isAcceptingApplications(app.posting):
211         raise ApiError(409, "POSTING_CLOSED", "접수 기간이 아닙니다.")
212     checkRequired(body.resume)
213     if body.method == "upload":
214         upload = db.get(ResumeUpload, body.upload_id) if body.upload_id else None
215         if upload is None or upload.user_id != user.id:
216             raise ApiError(400, "UPLOAD_REQUIRED", "올린 이력서 파일을 찾을 수 없습니다.")
217     resume = body.resume.model_dump()
218     app.resume_name = resume.pop("name")
219     app.resume_snapshot = resume
220     app.resume_method = body.method
221     app.edited_fields = body.edited_fields[:200]
222     app.attempt_status = st.transition(app.attempt_status, st.READY)
223     app.submitted_at = now()
224     criteria = currentCriteria(app.posting)
225     app.criteria_version = criteria.version if criteria else None
226     db.commit()
227     question_compose.scheduleCompose(app.id)
228     return {"attempt_status": app.attempt_status, "close_at": iso(app.posting.close_at), "release_at":
iso(app.posting.release_at)}
```

- **줄 204** 주소 속 `{applicationId}` = 🧑 **주소의 빈칸**. `/applications/abc123/resume` 이면 `applicationId` 에 `abc123` 이 들어가요.
- **줄 205** `async def` — 안에서 기다리는 일이 있을 수 있어서 비동기 자판기예요.
- **줄 206** ⭐ `ownedApplication(...)` (`deps.py:41`) — 🧑 "**이 지원이 정말 이 사람 것인가?**" 남의 것이면 404. `forUpdate=True` = 처리하는 동안 이 지원 줄을 잠가요.
- **줄 207** 작성 중일 때만 이력서를 낼 수 있어요 (이미 냈으면 잠김).
- **줄 208~211** 순서 지키기: 동의를 먼저 했나? 아직 접수 기간인가?
- **줄 212** 필수 칸(이름 등)이 비었는지 검사.
- **줄 213~216** "파일 올리기" 방식으로 냈다면, 그 파일이 진짜 있고 **이 사람이 올린 것인지** 확인.
- **줄 217** `model_dump()` — 🔧 Pydantic 객체 → 보통 사전(dict)으로.
- **줄 218** `resume.pop("name")` — 🧑 이름만 **꺼내서**(사전에서 빠짐) 따로 칸에 저장해요. 나머지가 `resume` 에 남아요.
- **줄 219** 이력서 사진(snapshot) 저장 (자물쇠 칸).
- **줄 221** 지원자가 고친 칸 목록. `[:200]` = 🧑 **최대 200개까지만** 잘라서. 누가 엄청 많이 보내도 창고가 터지지 않게.
- **줄 222** ⭐ 상태를 `ready` 로 (정류장 3 의 규칙판을 거침).
- **줄 224~225** 지금 공고의 평가 기준 **판(version) 번호**를 기억해요. 나중에 기준이 바뀌어도 이 지원은 이 판으로 채점해요.
- **줄 226** 확정!
- **줄 227** ⭐ 🧑 **질문 요리사에게 "이 사람 질문 만들기 시작해!"** 하고 쪽지를 넘겨요. 기다리지 않고 바로 다음 줄로 가요 (정류장 6).
- **줄 228** 화면에 상태와 마감·결과 공개 시각을 알려줘요. `iso(...)` = 시각을 `2026-11-11T10:00:00+09:00` 같은 표준 글자로.

✅ 체크

- [ ] `ownedApplication` (`deps.py:41`) 이 남의 지원을 404 로 숨기는 이유를 안다 (있는지조차 알려 주지 않으려고)
- [ ] 이력서를 올린 직후 화면이 계속 "추출 중" 을 묻는 방식(`extract-status` 반복 조회)을 이해했다

- [ ] `submitResume` 마지막 줄이 다음에 무슨 일을 시작시키는지 안다

정류장 6. 면접 질문 만들기 — 처음 만나는 AI

🧑 **쉬운 설명:** 질문 요리사는 "이 사람 이력서 + 이 공고" 를 보고 10개 질문을 만들어요. 그냥 상상으로 만들지 않고, **질문 책(질문은행)에서 비슷한 질문을 먼저 찾아 참고**한 다음 새로 써요. 비슷한 질문을 찾는 방법이 RAG 예요 — 글을 숫자 목록(벡터)으로 바꿔서 "숫자가 가까운 것 = 뜻이 비슷한 것" 으로 찾아요.

🔧 **전문 설명:** `compose` 가 예산 시간 안에서 LangGraph 그래프(`buildComposeGraph`)로 슬롯(인성·회사·이력서·직무)마다 RAG 예시 검색 → LLM 생성 → 검사(`checkGenerated`) → 실패 시 재생성 또는 질문은행/예비 질문으로 대체해요. LLM 호출은 전부 `llm.structured` 로 Pydantic 스키마에 맞춘 JSON 만 받아요.

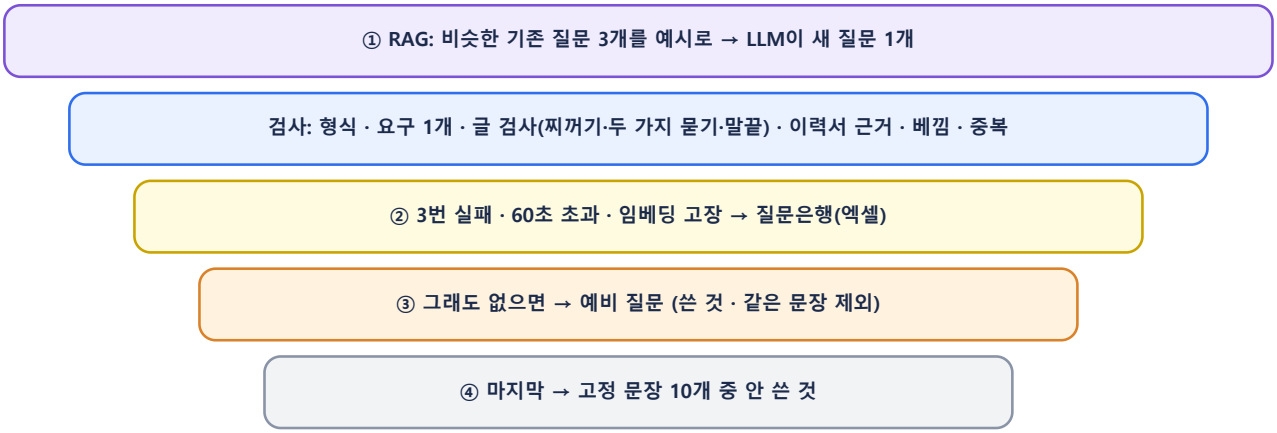


그림. 질문 만들기 안전망

👀 열어 볼 곳 (바깥에서 안으로)

순서	위치	무엇을 보나
①	`backend/app/services/question_compose.py:669`	`scheduleCompose` — 일을 맡기는 입구
②	`backend/app/services/question_compose.py:538`	`compose` — 재료 준비(`ctx` : 회사 소개, 이력서, 경험…). `fence(...)` 로 감싸는 이유 = 이력서 속 글이 AI 에게 명령처럼 읽히지 않게
③	`backend/app/services/question_compose.py:494`	`composeAll` — 슬롯을 돌며 하나씩 채움
④	`backend/app/services/question_compose.py:337`	`buildComposeGraph` — 한 슬롯을 만드는 순서도. 처음엔 노드 이름과 `add_edge` 만 보세요
⑤	`backend/app/services/llm.py:44`	`structured` — **모든 LLM 호출의 공통 입구**. 72줄 파일 전부 읽을 만해요
⑥	`backend/app/services/rag.py:77` , `:128`	`embed` (글→벡터), `similar` (비슷한 질문 찾기)
⑦	`backend/app/domain/question_text.py`	만들어진 질문 문장이 이상하지 않은지 검사하는 규칙

📖 코드 한 줄씩 읽기 — 쪽지 넘기기, AI 부르기

`services/question_compose.py:669` — 쪽지 넘기기

```
669  def scheduleCompose(applicationId: str) -> None:
670      if applicationId in runningTasks:
671          return
672      runningTasks[applicationId] = asyncio.get_running_loop().create_task(compose(applicationId))
```

- **줄 670~671** 🧑 이 사람 질문을 **이미 만들고 있으면** 또 시키지 않아요.
- **줄 672** `create_task(compose(...))` — 🧑 "요리 시작해 둬, 난 안 기다릴게." `compose` 는 따로 돌고, 이 함수는 바로 끝나요. `runningTasks` 사전에 "만드는 중" 으로 적어 둬요. 🥢 같은 이벤트 루프 안의 백그라운드 태스크예요. 워커(Celery)가 아니라 api 안에서 돌아요.

`services/llm.py:44` — 모든 AI 호출의 입구

```
44  async def structured(schema: type[BaseModel], system: str, user: str, timeoutSec: float,
45                       maxTokens: int = 400, temperature: float = 0.4, lane: str = "rt") -> BaseModel:
47      model = chatModel(timeoutSec, maxTokens, temperature, lane).with_structured_output(schema,
method="json_schema")
48      try:
49          result = await asyncio.wait_for(
50              model.ainvoke([SystemMessage(content=system), HumanMessage(content=user + NO_THINK)]),
51              timeout=timeoutSec,
52          )
53      except TimeoutError as exc:
54          raise LlmError("timeout") from exc
55      except Exception as exc:
57          log.warning("llm call failed (lane=%s): %s %s", lane, type(exc).__name__, ...)
58          raise LlmError(type(exc).__name__) from exc
59      if not isinstance(result, schema):
60          raise LlmError("schema")
61      return result
```

재료 설명

재료	🧑 뜻
`schema`	"**이 모양**으로만 대답해" 라는 빈칸 양식. 예: `{question: 글자, reason: 글자}`
`system`	AI 에게 주는 **역할 설명**. "너는 면접관이다. 규칙은..."
`user`	이번에 시킬 **구체적인 일**. "이 이력서로 질문 하나 만들어"
`timeoutSec`	이 시간 넘으면 포기
`maxTokens`	대답 최대 길이 (토큰 ≈ 글자 조각)
`temperature`	🧑 **상상력 손잡이**. 0 이면 늘 같은 대답, 높을수록 다양. 채점은 0, 질문 만들기는 조금 높게
`lane`	`rt` = 빠른 4B 요리사(GPU), `eval` = 꼼꼼한 8B 요리사(CPU)

- **줄 47** 요리사 연결을 만들고(`chatModel`), `.with_structured_output(schema, …)` 로 🧑 **"양식 빈칸만 채워서 내!"** 라고 묶어요. 🥢 llama.cpp 의 OpenAI 호환 API 에 JSON Schema 를 넘겨 출력 문법을 강제해요. LangChain 이

결과를 Pydantic 객체로 바꿔줘요.

- **줄 49~52** `ainvoke([역할, 일])` 로 부르고, `wait_for(…, timeout=)` 로 🧑 **알람을 맞춰 둬요.** 시간이 지나면 끊어요.
  - `NO_THINK` = `" /no_think"` — 🔗 Qwen3 모델에게 "속으로 길게 생각하는 단계는 건너뛰어" 라는 신호. 빨라져요.
- **줄 53~58** 어떤 실패든 `LlmError` **한 가지 이름**으로 바꿔 던져요. 쓰는 쪽은 `except LlmError` 하나만 준비하면 돼요.
  - **줄 57** 로그에는 오류 **종류 이름만** 남겨요. 🧑 오류 메시지 안에 지원자 답변이 섞여 있을 수 있어서, 개인정보가 로그에 남지 않게요 (CLAUDE.md 4.3).
- **줄 59~60** 혹시 모양이 다르면 또 거절. **두 번 검사**해요.
- **줄 61** 양식이 채워진 객체를 돌려줘요. 쓰는 쪽은 `out.question` 처럼 바로 꺼내 써요.

🔗 **왜 이렇게?** 자유 글로 받으면 "질문은 다음과 같습니다: …" 처럼 매번 모양이 달라 파싱이 깨져요. 스키마 강제 + 재검사 + 시간 제한 + 한 가지 오류 — 이 넷이 AI 를 **믿을 수 있는 부품**으로 만들어요.

✅ **체크**

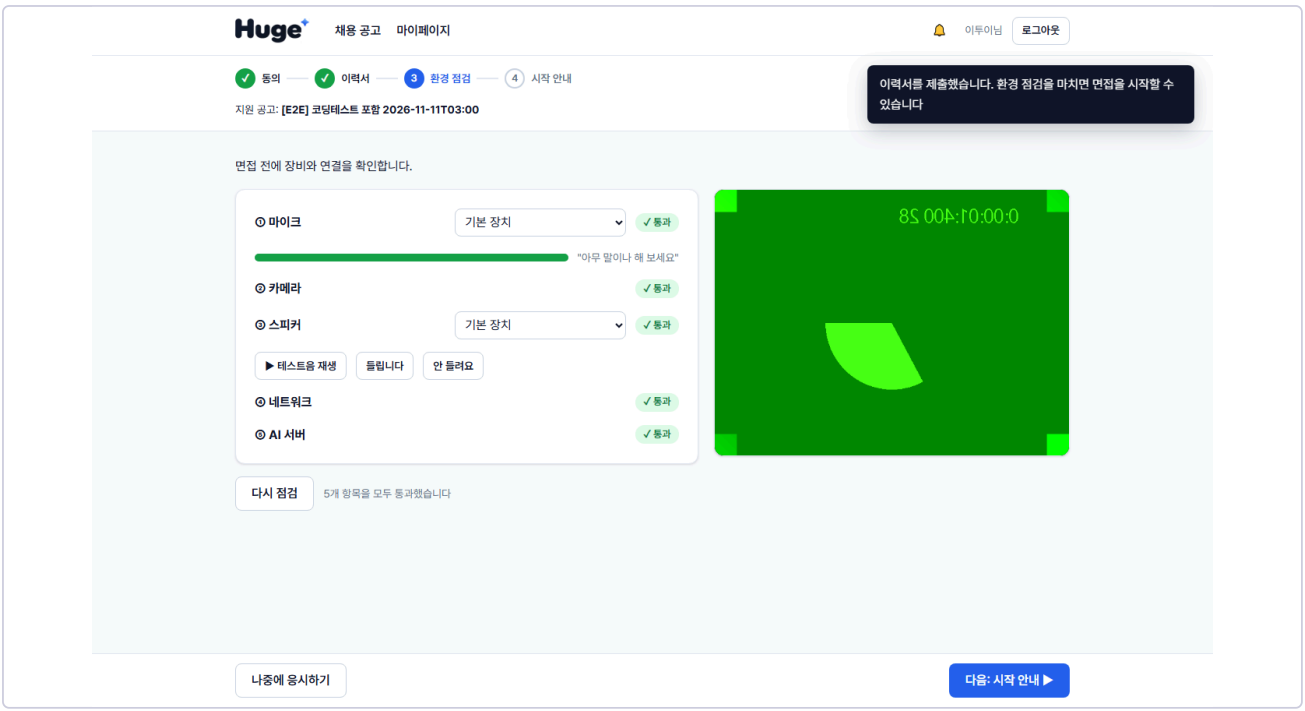
- [ ] AI 가 실패하거나 너무 느릴 때 질문이 0개가 되지 않는 이유를 말할 수 있다 (힌트: `bankFallback`, `lastResort`)
- [ ] `llm.structured` 가 "자유 글" 이 아니라 "정해진 모양의 JSON" 을 받는 장점을 말할 수 있다
- [ ] RAG 를 "숫자로 바꿔 가까운 것 찾기" 라고 한 문장으로 설명할 수 있다

정류장 7. 면접 진행 — 프로그램의 심장 💔

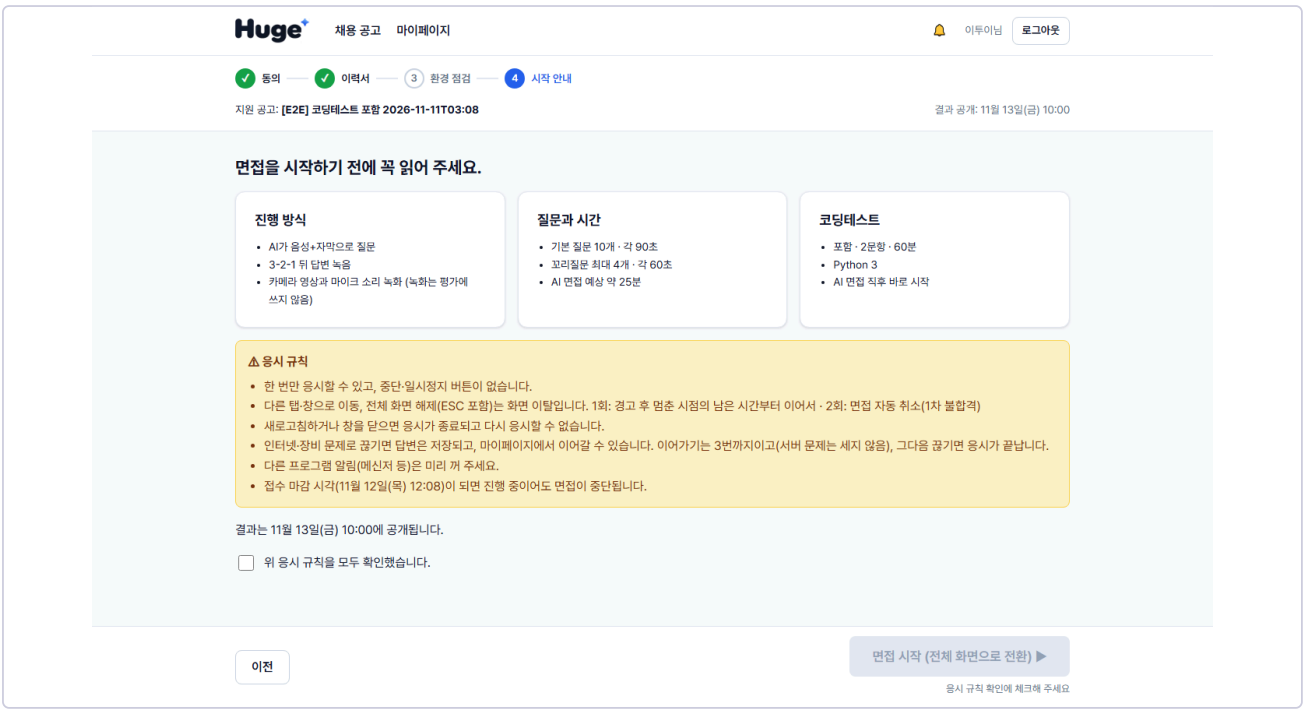
가장 길고 어려운 정류장이에요. 천천히, 두세 번에 나눠 보세요.

🧑 **쉬운 설명:** 면접은 **전화 통화**와 같아요. 보통 요청은 "편지 한 장 보내고 답장 한 장" 이지만, 면접은 서로 계속 말을 주고받아야 하니 **전화선(WebSocket)** 을 연결해 둬요. 서버 쪽에서는 `LiveSession` 이라는 면접관이 전화를 받고, **면접 진행표(LangGraph 그래프)** 를 보며 "질문 → 답 기다리기 → 받아쓰기 → 꼬리질문 할까? → 다음 질문" 을 반복해요. 진행표에는 책갈피(체크포인트)가 있어서 전화가 끊겨도 그 자리부터 이어 갈 수 있어요.

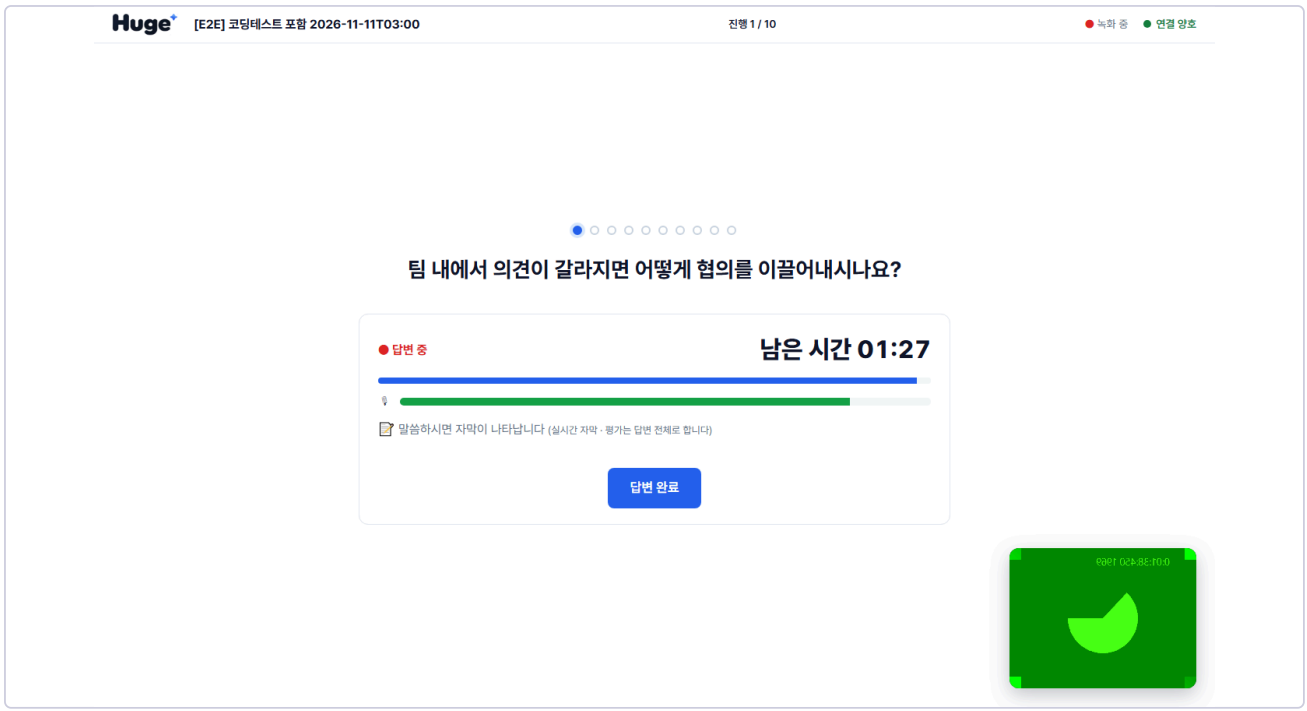
🔗 **전문 설명:** `interview/start` 가 세션 행을 만들고 `ws_url` 을 돌려줘요. 브라우저는 마이크 소리를 16kHz PCM 조각으로 WebSocket 에 보내고, 서버는 JSON 메시지(`question`, `caption`, `warn`, `phase`, `terminate` …)로 답해요. `LiveSession.runInterview` 가 그래프를 `ainvoke` 하다가 `wait` 노드의 `interrupt` 에서 멈추면 `collectAnswer` 로 답을 받고 `Command(resume=...)` 로 재개해요. 체크포인트는 `AsyncPostgresSaver`, thread_id 는 지원 번호예요.



화면. 환경 점검 (Precheck.tsx)



화면. 시작 안내 (apply/.../rules)



화면. 면접 중 — 이 화면과 engine.py 가 WebSocket 으로 이야기한다



그림. 편지(HTTP)와 전화(WebSocket)

👀 따라갈 길 — 화면 쪽

순서	위치	무슨 일
①	`frontend/components/Precheck.tsx`	환경 점검(마이크·카메라·스피커·인터넷). `frontend/lib/media.ts` 도구를 씀
②	`frontend/app/apply/[applicationId]/rules/page.tsx:48`	규칙 확인 후 `interview/start` 호출
③	`frontend/app/exam/[applicationId]/page.tsx:323`	`new WebSocket(...)` — **전화 거는 줄**
④	`frontend/app/exam/[applicationId]/page.tsx:206`	`onMessage` — 서버 말을 종류별(`switch (m.type)`)로 처리. `case` 이름만 훑어보세요
⑤	`frontend/app/exam/[applicationId]/page.tsx:143`	`presentQuestion` — 질문 보여 주기 → 읽어 주기 → 카운트다운 → 녹음
⑥	`frontend/lib/pcm.ts`	`PcmRecorder` — 마이크 소리를 작은 조각으로 잘라 보냄

👀 따라갈 길 — 서버 쪽

순서	위치	무슨 일
①	`backend/app/routers/interview.py:108`	`startInterview` — 응시 1회 확인, 동시 인원 자리(`admission.tryAcquire`), `interviewing` 으로 전이
②	`backend/app/routers/interview.py:188`	`interviewSocket` — 전화 받기. 출처·쿠키·주인 확인 후 `LiveSession(...).serve()`
③	`backend/app/interview/engine.py:175`	`serve` — 전화기 앞에 앉아 계속 듣는 반복문. 소리(`onAudio`)와 글(`onText`)을 나눠 받음
④	`backend/app/interview/engine.py:569`	`runInterview` — 진행표를 돌리고, 멈추면 답을 받아 다시 돌리는 7줄. **꼭 이해하기**
⑤	`backend/app/interview/graph.py:127`	`buildInterviewGraph` — 진행표 본체. 아래 그림과 맞춰 보세요
⑥	`backend/app/interview/engine.py:646` → `:663` → `:684`	`beginQuestion`(질문 보내기) → `collectAnswer`(답 모으기) → `finishAnswer`(받아쓰기·저장)
⑦	`backend/app/services/speech.py:12`, `:24`	`transcribe`(STT 서버 부르기), `synthesize`(TTS 서버 부르기)
⑧	`backend/app/interview/graph.py:87`	`buildFollowupGraph` — 꼬리질문 작은 진행표. `engine.py:748` `followupDecide` 가 LLM 에게 "더 물어볼까?" 를 물어요
⑨	`backend/app/interview/lifecycle.py:73`, `:90`	`markInterrupted`(중단), `finish`(끝내기 + 채점 맡기기)

면접 진행표 그림 (`graph.py:127` 을 그림으로)

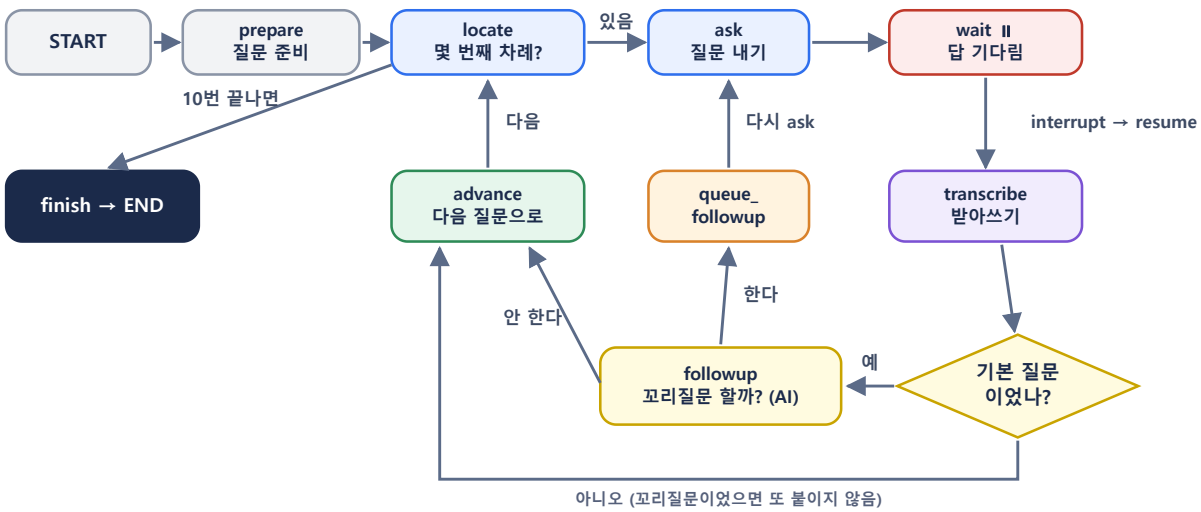


그림. 면접 진행표 (graph.py buildInterviewGraph)

📖 코드 한 줄씩 읽기 — 전화 받기부터 답 저장까지

① 전화 받기: `routers/interview.py:187`

```
187  @router.websocket("/ws/interview/{sessionId}")
188  async def interviewSocket(ws: WebSocket, sessionId: str):
189      if ws.headers.get("origin") != ENV.publicOrigin:
```

```
190         await ws.close(code=WS_CLOSE_UNAUTHORIZED)
191         return
192     db = SessionLocal()
193     try:
194         user = resolveSession(db, ws.cookies.get(COOKIE_NAME))
195         row = db.get(InterviewSession, sessionId)
196         app = db.get(Application, row.application_id) if row else None
197         ok = (user is not None and row is not None and row.user_id == user.id and row.ended_at is None
198               and app is not None and app.attempt_status in st.IN_PROGRESS)
199     finally:
200         db.close()
202     if not ok:
203         await ws.accept()
204         await ws.close(code=WS_CLOSE_UNAUTHORIZED)
205         return
206     if not registry.claimSession(sessionId):
207         await ws.accept()
208         await ws.close(code=WS_CLOSE_ALREADY_USED)
209         return
210     await ws.accept()
211     await LiveSession(ws, app.id, sessionId).serve()
```

- **줄 187** `@router.websocket` — 편지 창구(`post` / `get`)가 아니라 **전화 창구**.
- **줄 189~191** 🧑 "**우리 홀에서 건 전화가 맞나?**" 다른 사이트가 몰래 건 전화면 끊어요. 🔗 WebSocket 은 CORS 보호가 없어서 Origin 헤더를 직접 검사해요.
- **줄 192~200** 전화에는 `Depends` 문지기를 못 써서 **직접** 확인해요.
  - 팔찌(쿠키)로 누군지 (`resolveSession`)
  - 이 면접 연결이 **이 사람 것**이고, **아직 안 끝났고**, 지원이 **면접 중/코딩 중**인지
  - `finally` 로 창구는 반드시 닫아요.
- **줄 202~205** 아니면 끊어요. 🧑 **받고 나서 끊는** 이유: 받기 전에 끊으면 화면은 "그냥 끊김" 만 알아요. 받고 나서 "권한 없음" 번호를 말해 주고 끊어야 화면이 이유를 알아요.
- **줄 206~209** 🧑 **같은 연결 번호로 전화가 두 번 오면** 두 번째는 끊어요 ("이미 사용 중").
- **줄 210~211** 전화를 받고 → **면접관 `LiveSession` 에게 넘겨요.** `serve()` 가 끝날 때까지(면접이 끝날 때까지) 여기서 기다려요.

② 면접관이 전화기 앞에 앉기: `interview/engine.py:175` `serve`

```
175 async def serve(self) -> None:
176     previous = registry.get(self.appId)
177     if previous is not None:
178         await previous.terminateFromOutside("ended", "replaced")
179     registry.register(self.appId, self)
...
184     with dbSession() as db:
185         phase = db.get(Application, self.appId).phase
186     runner = self.runCoding() if phase == "coding" else self.runInterview()
187     self.tasks = [asyncio.create_task(self.guard(runner)), asyncio.create_task(self.watchdog()),
188                   asyncio.create_task(self.captionRelay())]
189     try:
190         while True:
191             message = await self.ws.receive()
192             if message["type"] == "websocket.disconnect":
193                 break
```

```
194             if message.get("bytes") is not None:
195                 await self.onAudio(message["bytes"])
196             elif message.get("text") is not None:
197                 await self.onText(message["text"])
198     except WebSocketDisconnect:
199         pass
200     finally:
202         if self.state != "ended":
203             await self.judgeAfterDisconnect()
204         await self.shutdown()
```

- **줄 176~179** 🧑 같은 지원자의 **옛날 면접관이 아직 있으면 퇴근**시키고, 내가 새 담당으로 등록해요. (탭 두 개로 동시에 면접 보는 걸 막아요)
- **줄 184~186** 면접 단계인지 코딩 단계인지 보고 할 일을 골라요.
- **줄 187~188** ⭐ 🧑 **일 세 개를 동시에** 시작해요.
  - `runner` = 면접 진행 (질문 내고 답 받기)
  - `watchdog` = 🐕 **지키는 개**. 화면에서 소식이 너무 오래 없으면 끊긴 걸로 봐요.
  - `captionRelay` = 말하는 동안 **임시 자막**을 화면에 전달
  - `guard(...)` = 진행 중 예상 못 한 오류가 나도 깔끔하게 정리하게 감싸요.
- **줄 190~197** ⭐ 🧑 **전화기를 계속 귀에 대고 있는 반복문.**
  - `receive()` = 화면이 뭔가 보낼 때까지 기다려요.
  - `bytes`(소리 조각) → `onAudio` 가 답변 소리를 모아요.
  - `text`(글 메시지, 예: "답변 끝", "경고 확인", "화면 이탈") → `onText` 가 종류별로 처리.
  - 끊김(`disconnect`) → 반복 끝.
- **줄 200~204** 🧑 **어떻게 끝났든** 마지막에: 면접이 정상 종료가 아니면 `judgeAfterDisconnect` 로 "왜 끊겼나" 를 판단(15초 기다려 기록을 봄 — 일부러 나갔나, 인터넷 장애인가)하고, `shutdown` 으로 정리해요.

③ 진행표 돌리기: `engine.py:569` `runInterview` (7줄 — 가장 중요)

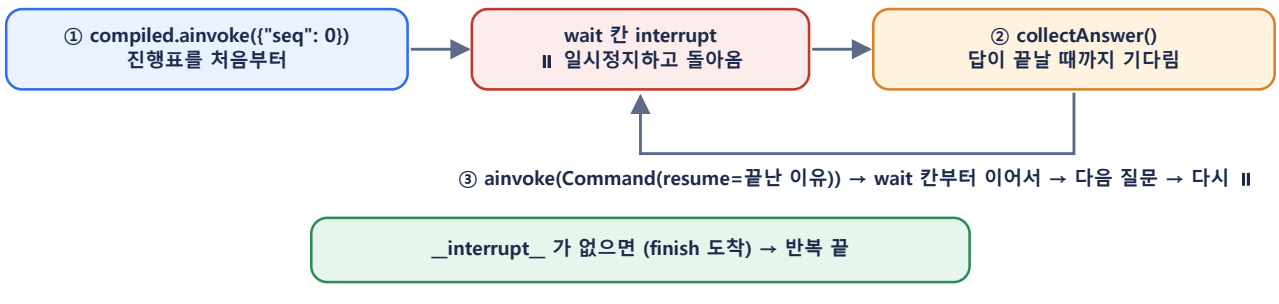


그림. runInterview 의 멈춤·이어가기 반복

```
569 async def runInterview(self) -> None:
571     compiled = graph.buildInterviewGraph(self, checkpoint.saver())
572     config = graph.graphConfig(self.appId)
573     result = await compiled.ainvoke({"seq": 0}, config)
574     while result.get("__interrupt__"):
575         endReason = await self.collectAnswer()
576         result = await compiled.ainvoke(Command(resume=endReason), config)
```

- **줄 571** 진행표(그래프)를 만들어요. 재료 두 개:

    - `self` = 🧑 진행표의 각 칸이 **실제로 일을 시킬 사람**(이 면접관). 진행표는 "질문해" 라고만 적혀 있고, 진짜 질문은 `self.beginQuestion` 이 해요.
    - `checkpoint.saver()` = 책갈피 보관함.
- **줄 572** `graphConfig(지원 번호)` — 🧑 **책갈피에 이름표**를 붙여요. 지원 번호별로 책갈피가 따로 있어요. 🔧 `{"configurable": {"thread_id": applicationId}}`.
- **줄 573** ⭐ 진행표를 **처음부터** 돌려요. `prepare → locate → ask → wait` 까지 가다가, `wait` 칸의 `interrupt` 에서 🧑 **"답 올 때까지 일시정지"** 하고 돌아와요. 이때 `result` 안에 `__interrupt__` 표시가 있어요.
- **줄 574** "일시정지 상태인 동안 계속 반복".
- **줄 575** 면접관이 **답이 끝날 때까지** 기다려요 (`collectAnswer`, ⑤). 끝난 이유(`done` 정상 / `timeout` 시간 초과)를 받아요.
- **줄 576** ⭐ `Command(resume=끝난이유)` — 🧑 **"일시정지 풀고 이어서!"** 진행표가 `wait` 칸부터 다시 가서 → 받아쓰기 → 꼬리질문 → 다음 질문 → 다시 `wait` 에서 멈춤. 마지막 질문까지 끝나면 `finish` 로 가서 `__interrupt__` 가 없어져 반복이 끝나요.

🧑 **책갈피 덕분에:** 인터넷이 끊겼다가 다시 연결하면, 새 면접관이 줄 573 부터 다시 시작해도 `locate` 칸이 **"몇 번 질문까지 답했지?"** 를 DB 에서 확인하고 그다음 질문부터 이어 가요.

④ 질문 내기: `engine.py:646` `beginQuestion`

```
646  async def beginQuestion(self, rowId: str) -> None:
648      with dbSession() as db:
649          app = db.get(Application, self.appId)
650          if app.attempt_consumed_at is None:
651              app.attempt_consumed_at = now()
652              criteria = currentCriteria(app.posting)
653              if criteria is not None:
654                  criteria.locked = True
655          row = db.get(InterviewQuestion, rowId)
656          row.asked_at = now()
657          db.commit()
658      await self.ackEvent.wait()
659      self.enter("answering")
660      self.current = {"row": row, "seq": row.seq, "answer_no": row.answer_order, "answer_sec":
row.answer_sec}
661      await self.sendQuestion(row)
```

- **줄 650~651** 🧑 **첫 질문이 나가는 순간 "응시 1회 사용"** 으로 찍어요. 환경 점검만 하고 나간 건 응시로 안 쳐요.
- **줄 652~654** 첫 응시가 시작되면 그 공고의 **평가 기준을 잠가요.** 누군가 응시한 뒤에 기준이 바뀌면 불공평하니까요.
- **줄 655~657** 이 질문을 "물어본 시각" 을 기록하고 확정.
- **줄 658** 🧑 화면에 **경고창이 떠 있으면 닫힐 때까지** 기다려요 (`ackEvent` = "확인 눌렀어요" 신호).
- **줄 659** 상태를 "답변 중" 으로.
- **줄 660** 지금 질문 정보를 기억 (몇 번째, 답변 제한 시간).
- **줄 661** 화면으로 질문 보내기 (`{"type": "question", …}` + TTS 소리 주소). 화면의 `onMessage` 의 `case "question"` 이 받아요.

⑤ 답 기다리기: `engine.py:663` `collectAnswer`

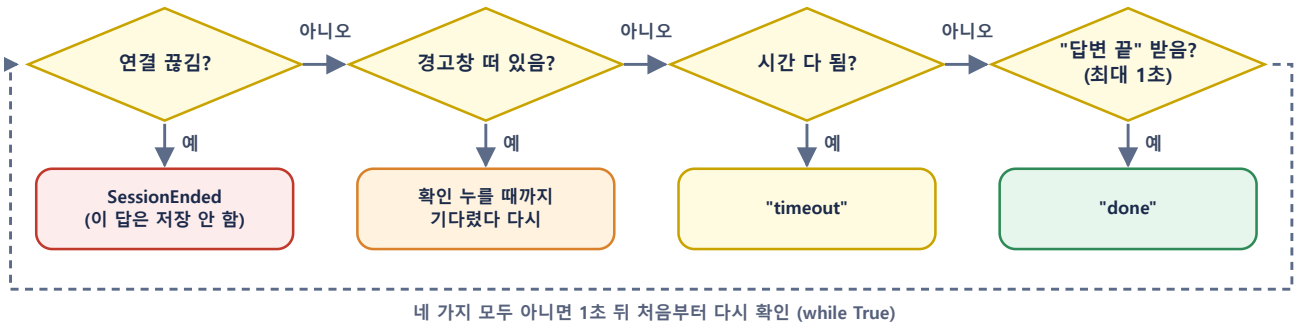


그림. collectAnswer 가 1초마다 하는 확인 네 가지

```
663  async def collectAnswer(self) -> str:
665      while True:
666          if self.state != "answering":
668              await self.endedEvent.wait()
669              raise SessionEnded()
670          if self.warned:
671              await self.ackEvent.wait()
672              continue
673          timeout = self.deadline - time.monotonic()
674          if timeout <= 0:
675              return "timeout"
676          try:
677              await asyncio.wait_for(self.answerEvent.wait(), timeout=min(timeout, 1.0))
678              if not self.warned:
679                  return "done"
680              self.answerEvent.clear()
681          except TimeoutError:
682              continue
```

🧑 **1초마다 시계를 보며 기다리는 반복문**이에요. 매번 네 가지를 확인해요.

줄	확인	그러면
666~669	연결이 끊겨서 "답변 중" 이 아니게 됐나?	판단이 끝날 때까지 기다렸다가 `SessionEnded` 로 멈춤. **이 답은 저장 안 함** (이어가기 때 같은 질문을 처음부터)
670~672	경고창(화면 이탈 경고)이 떠 있나?	확인 누를 때까지 기다렸다 다시 처음부터 확인
673~675	답변 시간이 다 됐나?	`"timeout"` 으로 끝
677~679	화면이 "답변 끝" 을 보냈나? (최대 1초 기다림)	`"done"` 으로 끝

- 🔧 `time.monotonic()` = 컴퓨터 시계를 누가 바꿔도 거꾸로 가지 않는 **스톱워치 시간**이에요.
- 🔧 `answerEvent` 는 `onText` 가 "답변 끝" 메시지를 받으면 켜는 신호등이에요. `wait_for(…, timeout=1.0)` 으로 최대 1초만 기다리고, 안 켜지면 `TimeoutError` → `continue` 로 다시 확인.

⑥ 받아쓰고 저장: `engine.py:684` `finishAnswer`

```
684  async def finishAnswer(self, rowId: str, endReason: str) -> dict:
686      self.enter("processing")
```

```
687     answeredAt = time.monotonic()
688     row = self.questionRow(rowId)
689     pcm = bytes(self.pcm)
690     transcript, words, sttSec = await self.transcribeWithRetry(pcm)
691     await self.saveAnswer(row, pcm, transcript, words, sttSec, endReason)
692     await self.send({"type": "caption", "seq": row.seq, "answer_no": row.answer_order, "chunk_no": 10_000,
693                      "text": transcript, "is_final": True})
694     return {"answeredAt": answeredAt, "sttSec": sttSec}
```

- **줄 686** 상태 "처리 중".
- **줄 687** 답 끝난 시각 기록 — 꼬리질문을 몇 초 안에 내야 하는지 계산할 때 써요.
- **줄 689** 그동안 `onAudio` 가 모아 둔 **소리 조각 전부**.
- **줄 690** ⭐🧑 **받아쓰기 요리사(STT)에게 소리를 주고 글자를 받아요.** 실패하면 한 번 더 시도(`WithRetry`). `words` = 낱말마다 몇 초에 말했는지 (나중에 하이라이트 재생용).
- **줄 691** 소리 파일 + 글자를 저장.
- **줄 692~693** 화면에 **최종 자막**을 보내요 (`is_final: True`).
- **줄 694** 진행표에 결과를 돌려줘요 → 다음 칸(꼬리질문 판단)으로.

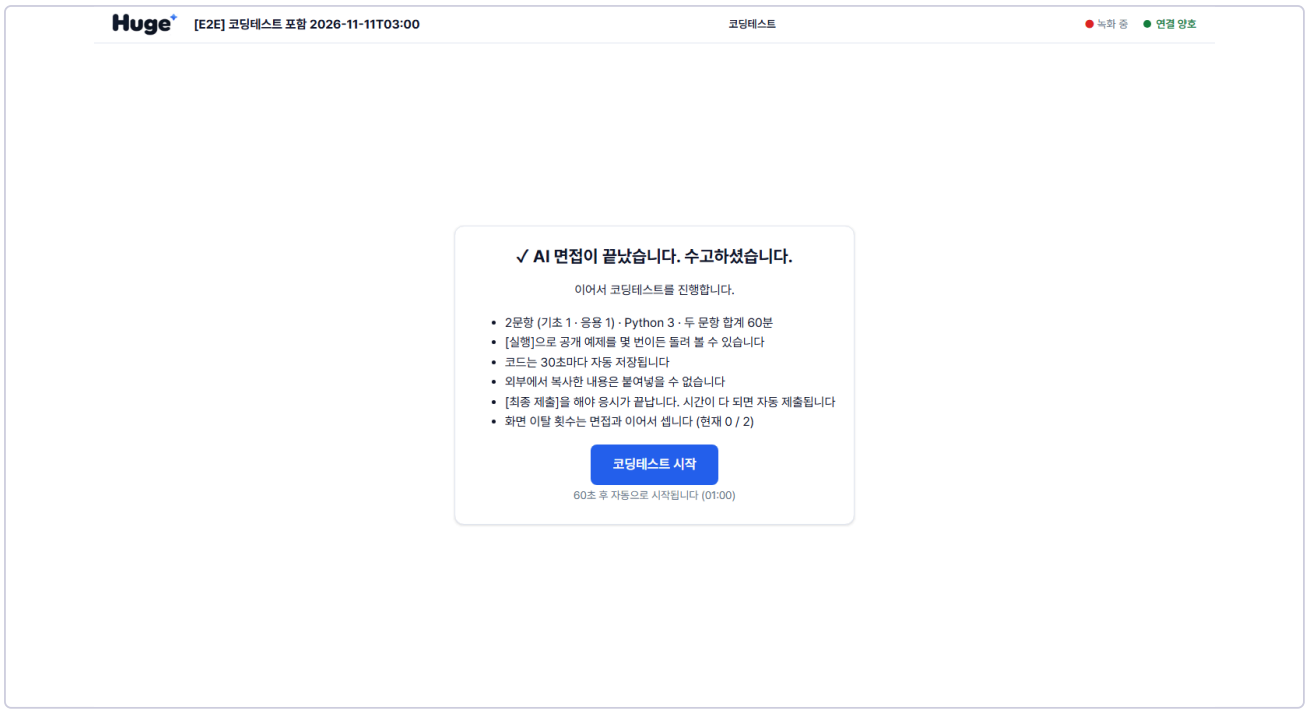
✅ **체크**

- [ ] 보통 요청(HTTP)과 WebSocket 의 차이를 편지/전화로 설명할 수 있다
- [ ] `interrupt` 에서 그래프가 멈췄다가 `Command(resume=...)` 로 다시 이어지는 흐름을 말할 수 있다
- [ ] 인터넷이 끊겼을 때 서버가 바로 "부정행위" 로 판단하지 않고 기록을 보고 판단하는 곳(`engine.py:236` `judgeAfterDisconnect` , `status.py:58` `judgeDisconnect`)을 찾았다
- [ ] 꼬리질문은 기본 질문 뒤에만 붙고, 꼬리질문 뒤엔 다시 안 붙는다는 걸 그림에서 찾았다

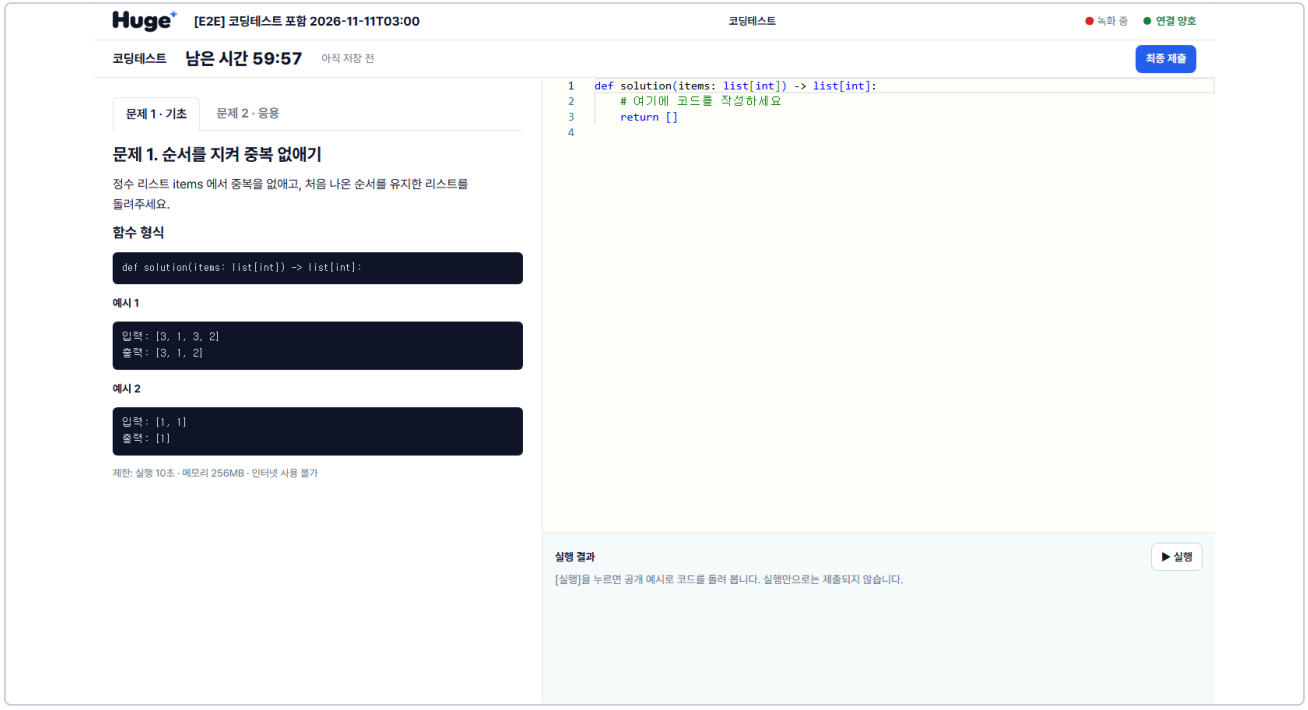
정류장 8. 코딩테스트 — 격리된 놀이방

🧑 **쉬운 설명:** 지원자가 쓴 코드는 무엇이 들어 있을지 몰라요. 그래서 **바깥과 연결이 끊긴 작은 놀이방(컨테이너)** 에 넣고 돌린 뒤, 놀이방째로 치워요.

🔧 **전문 설명:** 공고에 코딩테스트가 켜져 있으면 면접 뒤 `enterCoding` 으로 `coding` 단계에 들어가요. 실행은 api 가 직접 하지 않고 Celery `worker-sandbox` 가 네트워크 없는 일회용 컨테이너에서 돌려요(유일하게 `docker.sock` 접근). 붙여넣기 차단·자동 저장·시간 초과 자동 제출이 있어요.



화면. 면접이 끝나고 코딩테스트 안내 (lifecycle.enterCoding)



화면. 코딩테스트 (Coding.tsx)

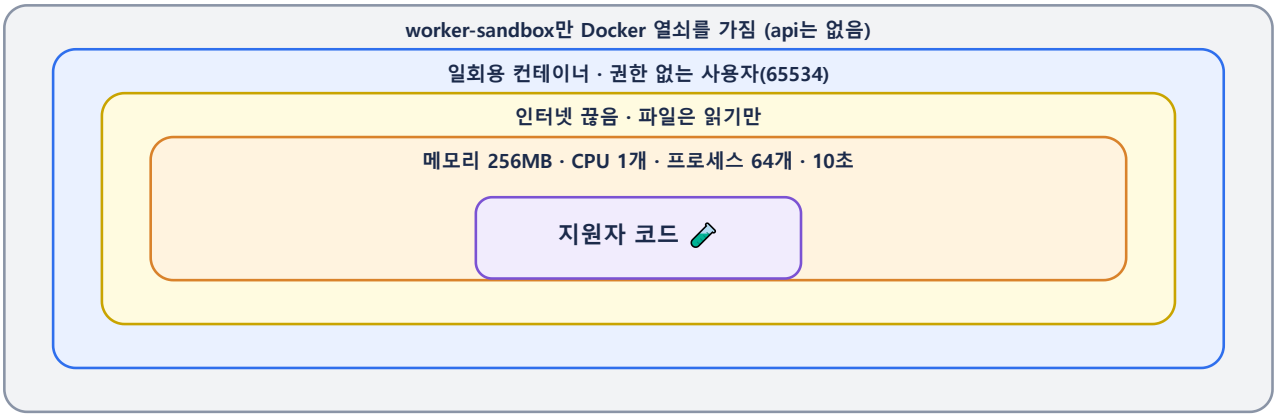


그림. 코드 실행 상자의 겹겹 보호

👀 열어 볼 곳

순서	위치	무슨 일
①	`backend/app/interview/lifecycle.py:110`	`enterCoding` — 면접 → 코딩 단계로
②	`frontend/components/exam/Coding.tsx`	코딩 화면 (Monaco 편집기)
③	`backend/app/routers/interview.py:258`	`codingRun` — [실행] 버튼
④	`backend/app/services/coding.py:29`	`runExamples` — 예시 입력으로 돌려 보기를 워커에 맡김
⑤	`backend/app/services/sandbox_runner.py:116`	`run` — 놀이방 만들고, 돌리고, 치우기. `judge` (77줄) 가 정답 비교
⑥	`backend/app/routers/interview.py:296`	`codingSubmit` — 제출

📖 코드 한 줄씩 읽기 — 놀이방 만들기: `services/sandbox_runner.py:116`



그림. 놀이방 규칙표

```
116  def run(code: str, cases: list[tuple[str, str]]) -> dict:
117      if not _slots.acquire(timeout=5):
118          raise SandboxBusy()
...
121      mark = "__HUGE_" + secrets.token_hex(8) + "__"
123          client = docker.from_env()
124          payload = {"code": code, "args": [args for args, _ in cases], "mark": mark}
125          container = client.containers.create(
126              IMAGE,
127              command=["python", "-I", "-c", HARNESS],
128              environment={"HUGE_PAYLOAD": base64.b64encode(json.dumps(payload).encode()).decode()},
```

```
129             network_disabled=True,
130             read_only=True,
131             user="65534:65534",
132             mem_limit="256m",
133             memswap_limit="256m",
134             nano_cpus=1_000_000_000,
135             pids_limit=64,
136             cap_drop=["ALL"],
137             security_opt=["no-new-privileges"],
138             tmpfs={"/tmp": "size=16m"},
...
142         container.start()
144             exit_info = container.wait(timeout=TIME_LIMIT_SEC)
...
147         if exit_info.get("StatusCode") == 137:
148             return {"status": "memory_exceeded", "results": []}
149         output = readLogs(container)
150         marker = output.rfind(mark)
...
157         return judge(raw if isinstance(raw, dict) else {}, cases)
...
162     finally:
163         if container is not None:
165                 container.remove(force=True)
```

- **줄 117~118** 🧒 **놀이방 개수 제한.** 동시에 너무 많이 만들면 컴퓨터가 힘들어서, 자리가 없으면 5초 기다리다 "바빠요" 로 거절.
- **줄 121** 🧒 **비밀 암호표**를 매번 새로 만들어요. 놀이방 출력 중 "이 암호 뒤가 진짜 결과" 라는 표시예요. 지원자 코드가 가짜 결과를 출력해도 암호를 모르니 속일 수 없어요.
- **줄 124** 놀이방에 넣을 짐: 지원자 코드 + 시험 입력들 + 암호.
- **줄 125~138** ⭐ **놀이방 규칙표** — 하나하나가 안전장치예요.

설정	🧒 뜻
`python -I`	파이썬을 "격리 모드" 로 (주변 파일을 몰래 불러오지 않음)
`environment=…base64…`	짐을 안전한 글자로 포장해서 전달
`network_disabled=True`	**인터넷 선 없음.** 밖으로 정보를 보낼 수 없어요
`read_only=True`	놀이방 벽에 낙서 금지 (파일 쓰기 불가)
`user="65534:65534"`	가장 힘없는 손님 계정으로 실행 (관리자 아님)
`mem_limit="256m"`	기억 공간 256MB 까지만
`nano_cpus=1_000_000_000`	CPU 1개 분량만
`pids_limit=64`	일꾼을 64명 넘게 못 불러요 (무한 복제 공격 방지)
`cap_drop=["ALL"]`	특별 권한 전부 뺏기
`no-new-privileges`	안에서 권한을 다시 얻을 수 없음
`tmpfs /tmp 16m`	낙서장은 16MB 짜리 임시 공책 하나만

- **줄 144** 정해진 시간만큼만 기다려요. 넘으면 → `timeout`.
- **줄 147~148** 종료 번호 137 = 🔧 기억 공간 초과로 강제 종료 (OOM kill).

- **줄 149~150** 놀이방 출력에서 **마지막 암호 위치**를 찾아 그 뒤만 결과로 읽어요.
- **줄 157** `judge` 가 정답과 비교해 몇 개 통과했는지 계산.
- **줄 162~165** ⭐ 🧒 **무슨 일이 있어도 놀이방은 부숴요** (`finally`). 다음 사람은 깨끗한 새 방을 써요.

✅ **체크**

- [ ] api 서버가 지원자 코드를 직접 실행하지 않는 이유를 말할 수 있다
- [ ] 코딩테스트가 꺼진 공고에서는 이 정류장을 건너뛴다는 걸 안다 (CLAUDE.md 3.3)

정류장 9. 채점과 결과 — 뒷주방의 일

🧒 **쉬운 설명:** 채점은 오래 걸려요(1~3분). 손님을 세워 두지 않으려고 **주문표를 뒷주방에 넘기고(큐)**, 뒷주방 요리사(worker)가 천천히 채점해요. AI 는 "항목별로 몇 점인지와 그 근거 문장" 만 매기고, **합격·불합격은 AI 가 아니라 계산 규칙(총점 ≥ 기준점)** 이 정해요.

🔧 **전문 설명:** `lifecycle.finish` → `queueEvaluation` → `worker.enqueueEvaluation` 으로 Celery 작업이 들어가요. `evaluateTask` 가 Redis 잠금을 잡고 `evaluation.evaluate` 를 실행. 문항별 루브릭 채점(T5, `scoreQuestion`) → `computeScores`(domain/scoring.py 의 가중치·반올림) → 합불 결정 → 사유(T10)·요약(T6)·피드백 → `save`. AI 서버가 죽어 있으면 나중에 다시 시도하고, 답변 속에 채점 지시가 보이면 `needs_review` 로 사람에게 넘겨요. 결과는 공개 시각 뒤에만 지원자에게 보여요.

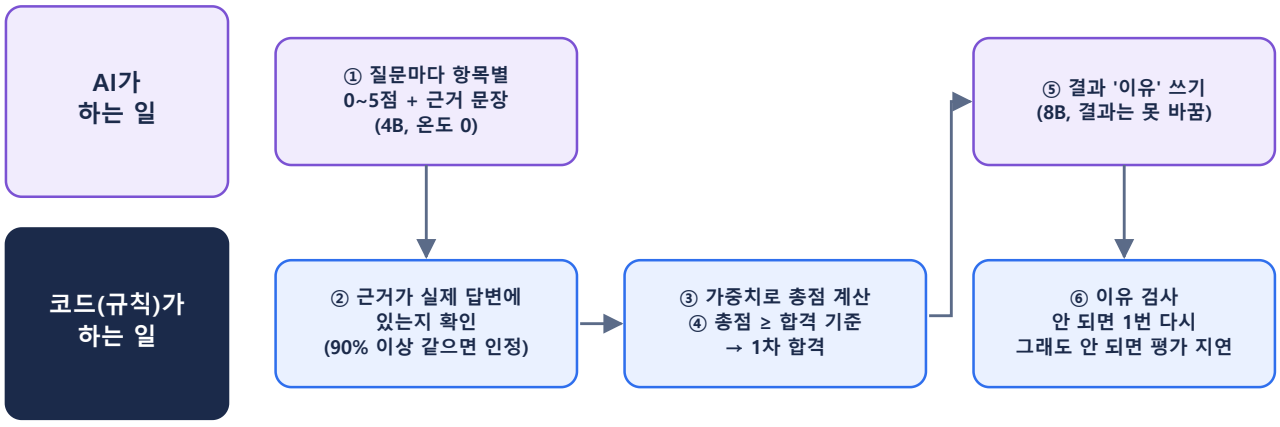


그림. AI 와 코드의 역할 나누기

👀 **따라갈 길**

순서	위치	무슨 일
①	`backend/app/interview/lifecycle.py:90`, `:103`	`finish` → `queueEvaluation` — 주문표 넘기기
②	`backend/app/worker.py:83`	`enqueueEvaluation` — 큐에 넣기
③	`backend/app/worker.py:165`	`evaluateTask` — 뒷주방이 받아 처리. 실패하면 `reschedule` 로 나중에 다시
④	`backend/app/services/evaluation.py:563`	`evaluate` ⭐ — 채점 전체 순서가 한 화면에 보여요. 끝까지 읽어요
⑤	`backend/app/services/evaluation.py:256`	`scoreQuestion` — 문항 하나를 AI 에게 채점받기

순서	위치	무슨 일
⑥	`backend/app/domain/scoring.py`	134줄 전부. `itemScores` → `interviewScore` → `codingScore` → `totalScore` 순으로. **AI 없이 순수 계산**
⑦	`backend/app/domain/scoring.py:96`	`matchQuote` — AI 가 준 근거가 실제 답변 속에 있는지 확인 (근거 조작 방지)
⑧	`backend/app/services/evaluation.py:611`	`save` — 결과 저장
⑨	`backend/app/routers/me.py:62`	`myResult` — 지원자가 결과 보기. `isReleased` (25줄)로 공개 시각 확인
⑩	`frontend/app/mypage/results/[applicationId]/page.tsx`	결과 화면

📖 코드 한 줄씩 읽기 — 채점부터 결과까지

① 뒷주방 요리사: `worker.py:165` `evaluateTask`

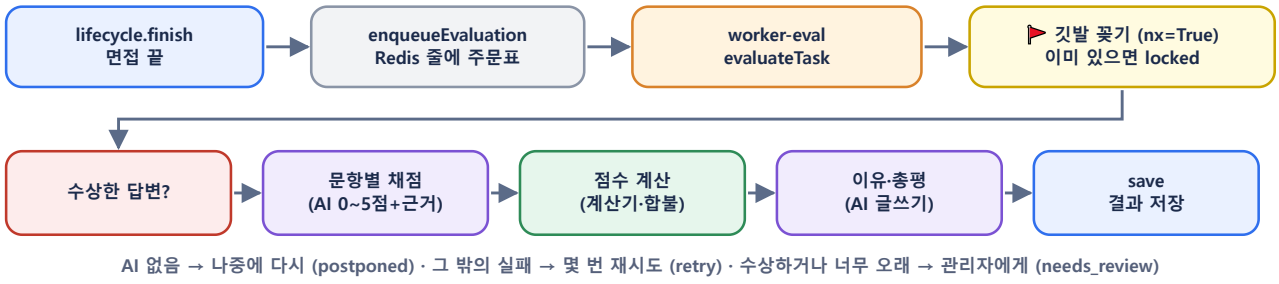


그림. 면접 끝 → 채점 → 저장

```
164  @celery.task(name=EVAL_TASK, bind=True)
165  def evaluateTask(self, applicationId: str, attempt: int = 1) -> str:
166      token = uuid.uuid4().hex
167      if not _redis.set(lockKey(applicationId), token, nx=True, ex=LOCK_TTL_SEC):
168          return "locked"
...
177      overdue = now() - evaluationStartedAt(applicationId, completedAt) >
timedelta(minutes=EVAL_DEADLINE_MIN)
178      try:
179          outcome = asyncio.run(evaluation.evaluate(applicationId))
180      except evaluation.NeedsReview as exc:
181          toNeedsReview(applicationId, exc.reason)
182          return "needs_review"
183      except evaluation.AiUnavailable as exc:
184          if overdue:
185              toNeedsReview(applicationId, "analysis_timeout")
186              return "needs_review"
188          reschedule(applicationId, attempt, AI_DOWN_DELAY_SEC)
189          return "postponed"
190      except Exception as exc:
192          if attempt > len(RETRY_DELAYS_SEC) or overdue:
193              toNeedsReview(applicationId, "analysis_timeout")
194              return "needs_review"
195          reschedule(applicationId, attempt + 1, RETRY_DELAYS_SEC[attempt - 1])
```

```
196         return "retry"
...
203     finally:
204         releaseLock(applicationId, token)
```

- **줄 164** `@celery.task` — 🧑 "이 함수는 **뒷주방 주문표**로 받을 수 있어요" 스티커. api 가 `enqueueEvaluation` 으로 주문표를 Redis 줄에 넣으면, `worker-eval` 컨테이너가 꺼내서 이 함수를 실행해요.
- **줄 166~168** ⭐ 🧑 **"이 지원 채점은 내가 맡았어!" 깃발 꽂기.**
  - `nx=True` = 깃발이 **없을 때만** 꽂혀요. 이미 다른 요리사가 꽂았으면 실패 → `"locked"` 로 그냥 돌아가요. 같은 사람을 두 번 채점하지 않아요.
  - `ex=LOCK_TTL_SEC` = 깃발은 일정 시간 뒤 저절로 사라져요 (요리사가 쓰러져도 영원히 막히지 않게).
  - `token` = 내 깃발 표시. 나중에 **내 깃발만** 뽑으려고요.
- **줄 177** 너무 오래(정해진 분 수) 끌었는지.
- **줄 179** ⭐ 진짜 채점 (`evaluate`, ②). 🔧 Celery 작업은 동기 함수라 `asyncio.run` 으로 비동기 함수를 실행해요.
- **줄 180~196** 실패 종류별로 다르게 대응 — 🧑 **실패에도 이름표가 있어요.**

실패	뜻	대응
`NeedsReview`	답변에 "만점 줘" 같은 지시가 있음	**사람(관리자)에게 넘김**
`AiUnavailable`	AI 요리사가 자리에 없음	나중에 다시 (`postponed`). 단 너무 오래됐으면 사람에게
그 밖의 모든 실패	이상한 대답 등	몇 번 다시 시도(`retry`), 다 써 버리면 사람에게

- **줄 203~204** 무슨 일이 있어도 **내 깃발 뽑기**.

② 채점 순서: `services/evaluation.py:563` `evaluate` (핵심만)

```
568         if app is None or app.attempt_status not in st.FINAL_STATUSES or app.attempt_status ==
st.NOT_TAKEN_CLOSED:
569             return "skipped"
570         if app.analysis_status == "needs_review" or
db.query(Result).filter_by(application_id=app.id).first() is not None:
571             return "skipped"
572         data = loadInput(db, app)
574         hits = instructionHits(data.questions) if data.attemptStatus != st.CANCELLED else []
575         if hits and not instructionAcknowledged(db, app.id, hits):
576             raise NeedsReview("instruction_suspected")
...
582     if codingWaiting(data):
583         return "waiting_coding"
...
587     if data.attemptStatus == st.CANCELLED:
588         scores = Scores({k: Decimal(0) for k in scoring.EVAL_ITEMS}, ...)
...
593     else:
594         if any(q.answer for q in data.questions):
595             for lane in ("rt", "eval"):
596                 if not await llm.health(lane=lane):
597                     raise AiUnavailable(f"llm-{lane}")
598         for q in data.questions:
599             perQuestion[q.seq] = await scoreQuestion(q, data)
600         scores = computeScores(data, perQuestion)
601         reason, decisionCheck = await writeReason(data, scores, perQuestion)
```

```
602          summary = await writeSummary(data, perQuestion, scores)
603          strengths, improvements = await writeFeedback(data, scores)
604          for problem in data.coding:
605              codingReviews[problem["problem_no"]] = await reviewCode(problem, data)
607      save(data, scores, perQuestion, reason, decisionCheck, summary, strengths, improvements,
codingReviews)
```

🧑 채점 요리 순서표

줄	단계	쉬운 말
568~571	채점할 대상인가?	응시가 안 끝났거나, 이미 결과가 있거나, 사람 확인 대기 중이면 건너뜀
572	재료 모으기	질문·답변·평가 기준·코딩 결과를 한 상자에 (`loadInput`)
574~576	수상한 답변 검사	답변에 "이 답은 100점" 같은 **채점 지시**가 있으면 사람에게 먼저
582~583	코딩 채점 기다리기	코딩 실행 결과가 아직이면 나중에
587~588	응시 취소면	전부 0점, AI 안 씀
594~597	AI 요리사 출근 확인	둘 다 있어야 시작
598~599	⭐ 문항마다 채점	AI 가 항목별 0~5점 + **답변 속 근거 문장**
600	⭐ 점수 계산	**AI 없이 계산기로** (③) — 여기서 합불이 정해져요
601	결과 이유 쓰기	8B AI 가 "왜 이 결과인지" 를 근거 문장으로
602~603	총평·강점·개선점	지원자에게 보여 줄 피드백
604~605	코드 리뷰	코딩 답안 짧은 평
607	저장	결과 서랍(`Result`)에

③ 계산기: `domain/scoring.py` — 숫자로 따라 해 보기

```
46  def itemScores(questionScores: list[dict[str, int]]) -> dict[str, Decimal]:
51      result = {}
52      for item in EVAL_ITEMS:
53          values = [Decimal(s[item]) for s in questionScores if item in s]
54          if not values:
55              result[item] = Decimal(0)
56              continue
57          result[item] = roundScore(sum(values) / len(values) / 5 * HUNDRED)
58      return result

61  def interviewScore(items: dict[str, Decimal], weights: dict[str, int]) -> Decimal:
63      return roundScore(sum(items[k] * Decimal(weights[k]) / HUNDRED for k in EVAL_ITEMS))

74  def totalScore(interview: Decimal, coding: Decimal | None, codingEnabled: bool, codingRatio: int) ->
Decimal:
76      if not codingEnabled:
77          return interview
78      ratio = Decimal(codingRatio)
79      return roundScore(interview * (HUNDRED - ratio) / HUNDRED + (coding or Decimal(0)) * ratio / HUNDRED)
```

🎛️ 예시로 계산해 봐요



그림. 점수 계산 따라 하기

1단계 — 항목 점수 (`itemScores`, 줄 46) "전문성(expertise)" 을 본 질문이 3개였고 AI 가 4점, 5점, 3점(5점 만점)을 줬다면:

- 줄 53: 전문성 점수만 모음 → `[4, 5, 3]` (`if item in s` = 전문성을 안 보는 인성 질문은 빠져요)
- 줄 57: 평균 4 → `÷ 5 × 100` → **80점**

2단계 — 면접 점수 (`interviewScore`, 줄 61) 항목 점수 × 비율(기본 전문성 30%, 논리 25%, 관련성 20%, 전달력 10%, 조직 적합 15%)을 모두 더해요.

항목	항목 점수	비율	곱
전문성	80	30%	24.00
논리	70	25%	17.50
관련성	90	20%	18.00
전달력	60	10%	6.00
조직 적합	80	15%	12.00
합			**77.50**

3단계 — 총점 (`totalScore`, 줄 74)

- 코딩테스트가 꺼진 공고 → 줄 76~77: 면접 점수가 그대로 총점 **77.50**
- 켜져 있고 코딩 비율 30%, 코딩 점수 50 이라면 → 줄 79: `77.50 × 70% + 50 × 30% = 54.25 + 15.00 =` **69.25**

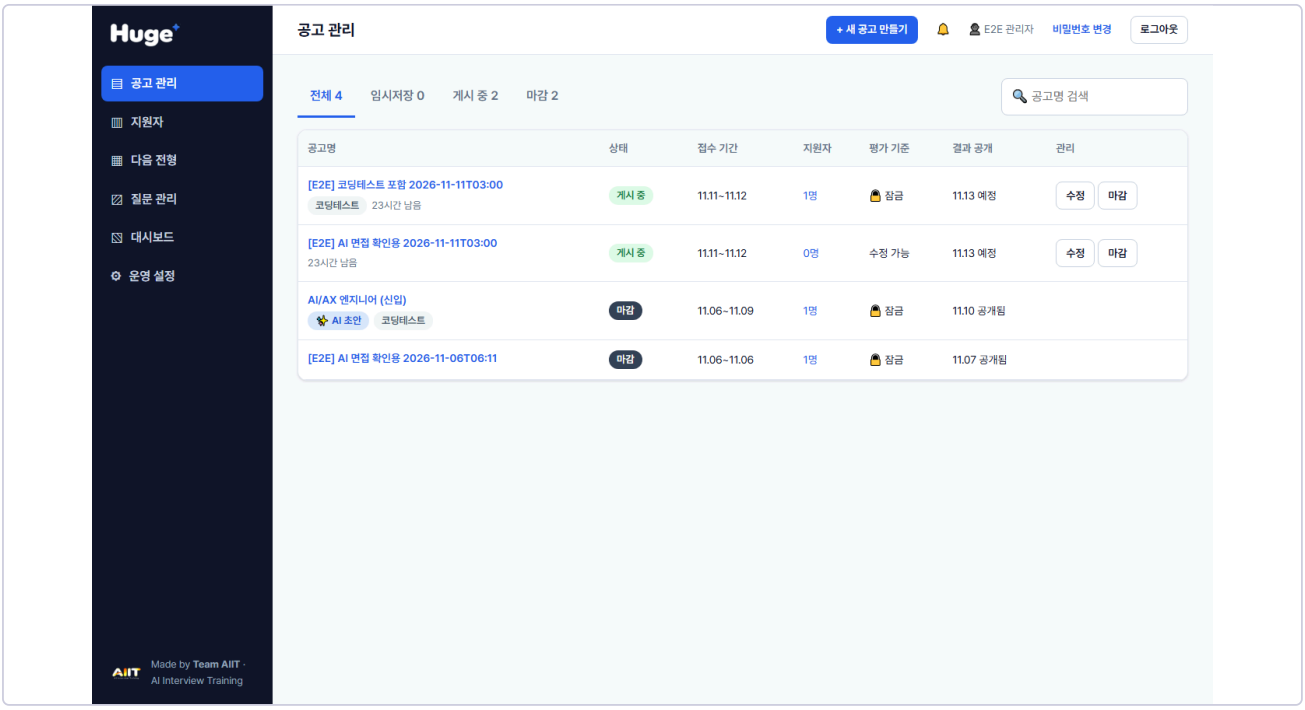
4단계 — 합불: 총점이 공고의 **기준점 이상이면 합격**. 🧑 AI 는 1단계의 0~5점과 근거만 주고, **합격·불합격은 이 계산기가 정해요.**

🪛 `Decimal` 은 `0.1 + 0.2 = 0.30000000000000004` 같은 소수 오차가 없는 숫자예요. 점수에 꼭 필요해요. `roundScore` 는 소수 둘째 자리 반올림(`ROUND_HALF_UP`, 우리가 아는 사사오입).

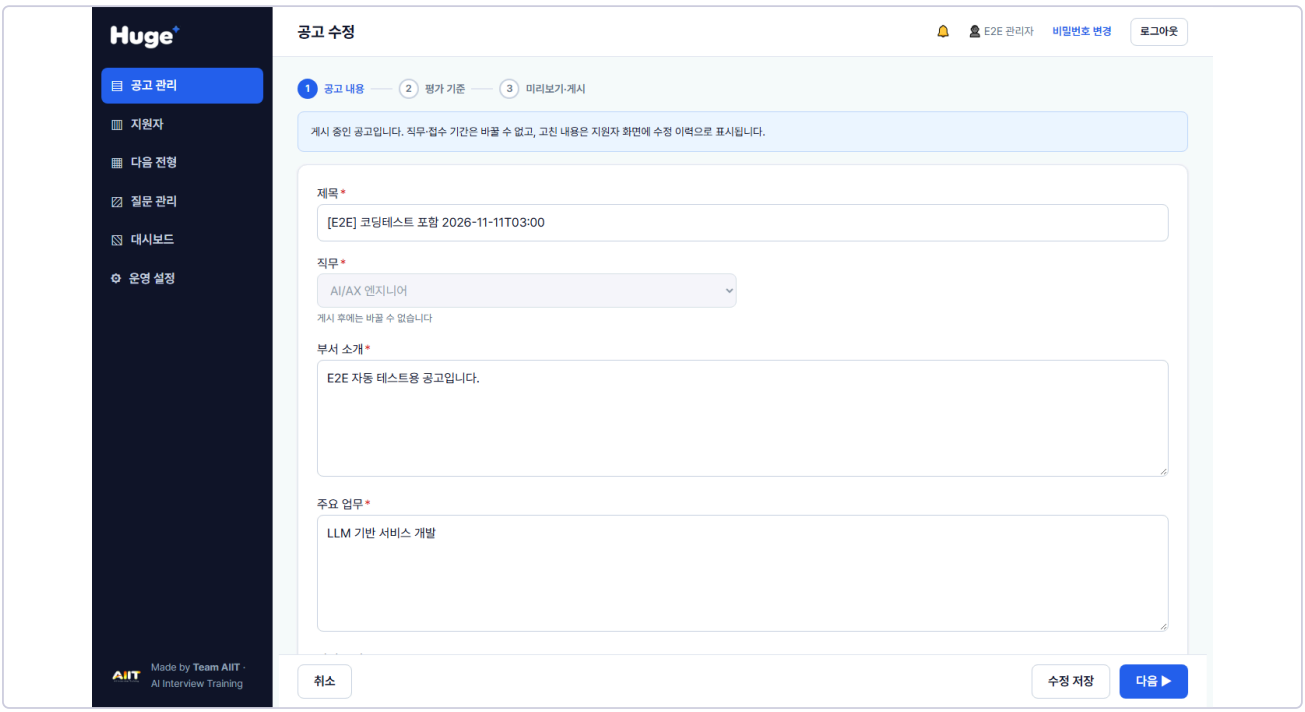
✅ 체크

- [ ] "AI 가 점수를 매기지만 합불은 코드가 정한다" 를 `evaluate` 안에서 찾아 확인했다
- [ ] 채점을 api 가 아닌 worker 가 하는 이유를 말할 수 있다
- [ ] 🧪 직접 해 보기: `backend/tests/test_scoring.py` 를 읽고 돌려 본다. 가중치를 바꾸면 총점이 어떻게 변하는지 테스트에서 찾는다

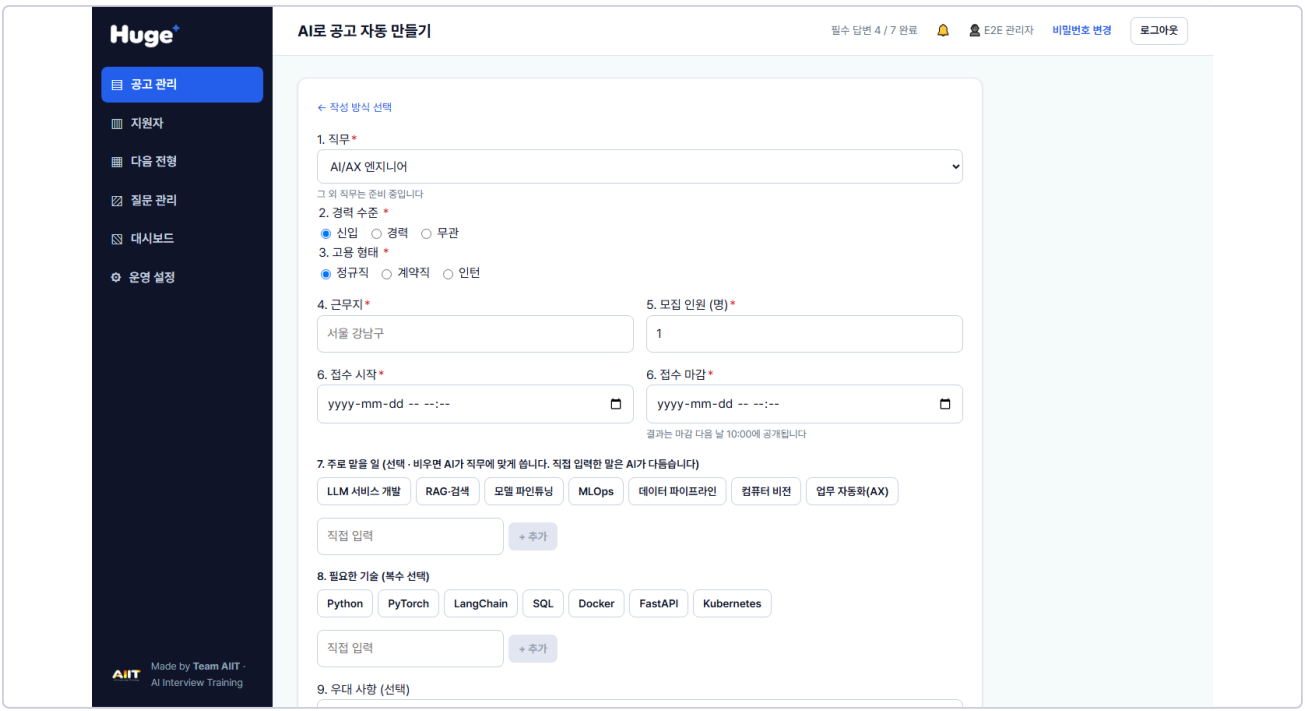
정류장 10. 관리자 화면 — 반대편 손님



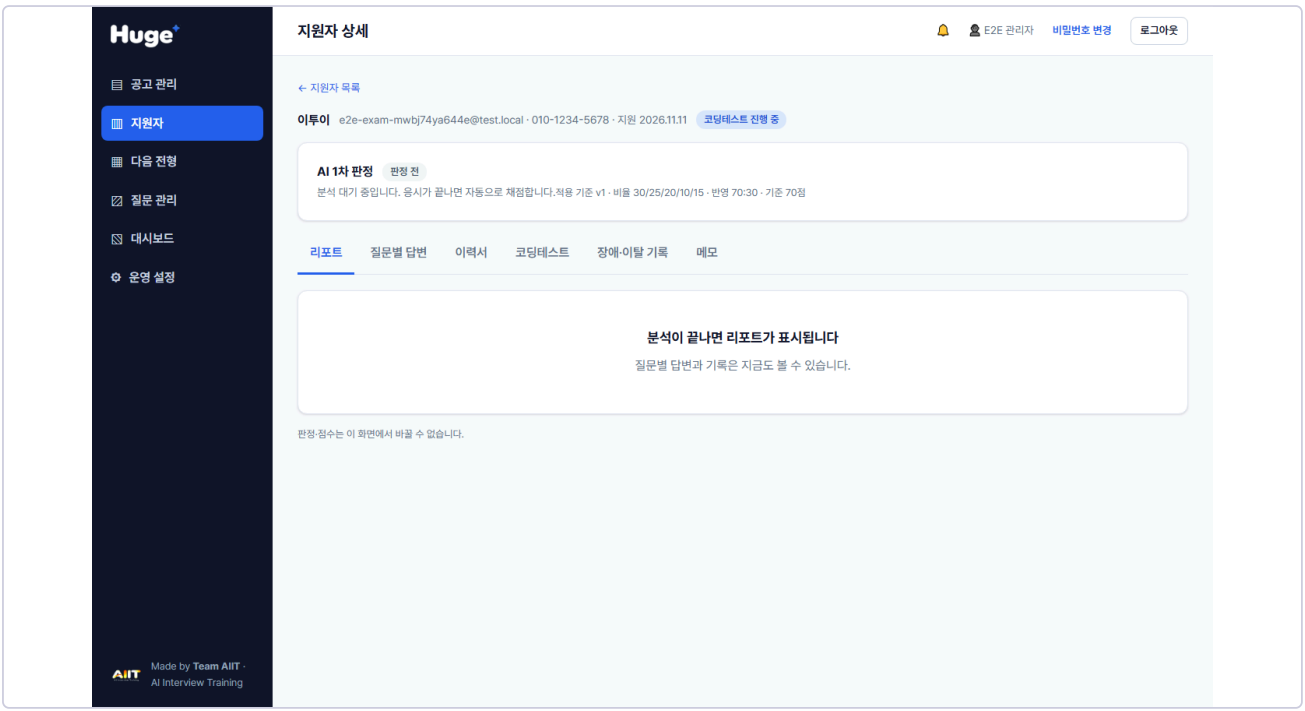
화면. 관리자 공고 목록 (admin/postings)



화면. 공고 수정 — 코딩테스트 토글과 평가 비율



화면. AI 공고 초안 (draftAi → posting_draft)



화면. 지원자 상세 (applicantDetail)



그림. 요청 하나가 지나가는 층 — 관리자 창구도 같은 모양

🧑 **쉬운 설명:** 지금까지는 지원자 쪽 길이었어요. 관리자는 **메뉴판(공고)을 만들고, 손님(지원자) 결과를 보고, 식당 규칙(설정)을 바꾸는** 사람이에요. 길의 모양(화면 → `api()` → 라우터 → 서비스)은 똑같아요. 문지기만 `requireAdmin` 이에요.

👀 열어 볼 곳

무엇	화면	서버
관리자 틀	`frontend/components/AdminShell.tsx`	—
공고 목록·작성	`frontend/app/admin/postings/page.tsx`, `frontend/components/PostingEditor.tsx`	`backend/app/routers/admin.py:48`, `:65`, `:82`
AI 공고 초안	`frontend/app/admin/postings/ai/page.tsx`	`admin.py:89` `draftAi` → `services/posting_draft.py:486` `start` → `:340` `buildDraftGraph` (또 하나의 LangGraph)
공고 게시	—	`admin.py:146` `publish` → `services/postings.py:164` `validateForPublish`, `services/posting_questions.py` (게시 때 질문 후보)
지원자 목록·상세	`frontend/app/admin/applicants/...`	`admin.py:194` `applicants`, `:246` `applicantDetail`
확인 필요 건 직접 결정	—	`admin.py:380` `manualDecision` → `evaluation.py:686` `decideManually`
질문 관리	`frontend/app/admin/questions/page.tsx`	`backend/app/routers/admin_questions.py`
운영 설정·대시보드·비교	`frontend/app/admin/settings`, `dashboard`, `compare`	`admin.py:402`, `backend/app/routers/admin_ops.py`, `backend/app/settings_store.py`

📖 코드 한 줄씩 읽기 — 스스로 해 보기

관리자 창구는 정류장 4·5 와 **똑같은 모양**(`@router` 스티커 → `Depends(requireAdmin)` 문지기 → 서랍 꺼내기 → 규칙 확인 → `commit` → 답장)이라 따로 풀어 쓰지 않았어요. 정류장 4·5 에서 읽은 방법을 `routers/admin.py` 에 그대로 써 보세요. 막히는 함수가 있으면 이름을 알려 주세요. 같은 방식으로 풀어 드릴게요.

✅ 체크

- [ ] 관리자 창구 함수들이 전부 `Depends(requireAdmin)` 을 쓰는지 훑어서 확인했다
- [ ] AI 공고 초안도 "화면이 계속 상태를 묻는 방식"(`draft-ai/status`)인 걸 알아챘다 (정류장 5 이력서 추출과 같은 모양)

정류장 11. 뒤에서 혼자 도는 일 — 시계 보는 일꾼

🧑 **쉬운 설명:** 손님이 아무것도 안 해도 식당은 돌아가요. "마감 시간 된 공고 닫기", "5분마다 빠진 채점 다시 넣기", "새벽 3시에 오래된 자료 버리기", "결과 공개 시각에 알림 보내기" 같은 일을 **시계를 보고** 해요.

🩹 **전문 설명:** 두 군데예요. ① api 안의 `jobs.runForever` (가벼운 정리: 자리 반납, 마감 처리, 자리 수 맞추기) ② Celery beat(`scheduler` 컨테이너)의 `beat_schedule` (sweep 5분, reminders 매시, purge 03:00, purgeNotice 09:00, purgeRecordings 매시 30분).



그림. 하루 동안 자동으로 도는 일

👀 열어 볼 곳

1. `backend/app/jobs.py:70` — `runForever` 반복문 (15줄)
2. `backend/app/worker.py:66` — `beat_schedule` 표. 이름과 시간만 보면 돼요
3. `backend/app/worker.py:304` — `sweepTask` (빠진 채점·결과 공개 처리)
4. `backend/app/services/retention.py` — 보관 기간이 지난 자료 삭제. 개인정보 원칙(CLAUDE.md 4.3)과 연결
5. `backend/app/services/notify.py` — 알림·메일

📖 코드 한 줄씩 읽기 — `jobs.py:70`

```
70  async def runForever() -> None:
71      while True:
72          db = SessionLocal()
73          try:
74              sweepOrphans(db)
75              releaseHeldSeats(db)
76              await closeExpiredPostings(db)
77              admission.reconcile(db)
78              regradePending()
79          except Exception:
80              log.exception("periodic job failed")
81              db.rollback()
82          finally:
83              db.close()
84          await asyncio.sleep(INTERVAL_SEC)
```

- **줄 71** `while True` — 🧑 **영원히 반복** (서버가 꺼질 때 `cancel` 될 때까지).
- **줄 74** 연결이 끊긴 채 방치된 면접 정리
- **줄 75** 장애로 붙잡아 둔 면접 자리(10분)가 지나면 반납
- **줄 76** 마감 시각이 지난 공고 닫기 (그 공고의 진행 중 응시도 마감 처리)
- **줄 77** 동시 면접 자리 수 맞추기
- **줄 78** 확인 필요 건 다시 채점 맡기기
- **줄 79~81** 🧑 하나가 실패해도 **반복은 멈추지 않아요.** 기록만 남기고 고친 것은 되돌려요(`rollback`).
- **줄 84** `INTERVAL_SEC = 60` → **1분 쉬고** 다시. 🪥 `asyncio.sleep` 은 쉬는 동안 다른 요청을 막지 않아요 (`time.sleep` 과 달라요).

✅ **체크**

- [ ] "누가 버튼을 안 눌러도 일어나는 일" 3개를 말할 수 있다
- [ ] 채점 작업이 한 번 빠져도 결국 다시 처리되는 이유를 안다 (5분 sweep)

정류장 12. 안전장치 — 문지기와 자물쇠

🧑 **쉬운 설명:** 지금까지 길을 걸으며 문지기(`Depends`)를 계속 만났죠. 여기서는 나머지 자물쇠들을 한데 모아 봐요.

자물쇠	위치	하는 일
오류 한 모양	`backend/app/errors.py` (31줄)	모든 오류를 `{code, message}` 한 모양으로. 예상 못 한 오류는 속 내용을 숨김
문지기 사다리	`backend/app/deps.py`	정류장 4 복습
로그인 막기	`backend/app/services/login_block.py`	같은 IP 에서 계속 틀리면 잠깐 막음
암호화	`backend/app/crypto.py:41` , `:51`	민감한 칸은 DB 에 암호문으로 저장(`EncryptedText`)
큰 파일 막기	`backend/app/main.py:94`	`UploadSizeGuard` — 너무 큰 업로드는 받기 전에 거절
안전한 이동 주소	`backend/app/security.py:66`	`safeNext` — 로그인 뒤 이상한 바깥 주소로 안 보냄
탈퇴·동의 철회	`backend/app/services/withdrawal.py`	내 자료 지우기
프롬프트 감싸기	`evaluation.py:163` `fence` , `:180` `instructionHits`	답변 속 "만점 줘" 같은 말을 AI 가 명령으로 따르지 않게

📖 코드 한 줄씩 읽기 — 오류 쪽지 한 모양: errors.py (31줄 전부)

```
 8  class ApiError(Exception):
 9      def __init__(self, status: int, code: str, message: str):
10          self.status = status
11          self.code = code
12          self.message = message

15  def notFound() -> ApiError:
16      return ApiError(404, "NOT_FOUND", "찾을 수 없습니다.")

19  async def apiErrorHandler(_request: Request, exc: ApiError):
```

```
20      return JSONResponse(status_code=exc.status, content={"code": exc.code, "message": exc.message})

23  async def unexpectedErrorHandler(_request: Request, _exc: Exception):
25      return JSONResponse(status_code=500, content={"code": "SERVER_ERROR", "message": "잠시 후 다시 시도해
주세요."})

28  async def validationErrorHandler(_request: Request, exc: RequestValidationError):
29      first = exc.errors()[0] if exc.errors() else {}
30      field = ".".join(str(p) for p in first.get("loc", [])[1:])
31      return JSONResponse(status_code=422, content={"code": "VALIDATION_ERROR", "message": f"{field}:
{first.get('msg', '형식 오류')}"})
```

- **줄 8~12** 🧑 우리만의 **"문제 쪽지" 틀**. 쪽지에는 번호(`status`, 예: 409), 짧은 이름(`code`, 예: `ALREADY_APPLIED`), 사람이 읽을 문장(`message`) 세 칸이 있어요. 🔧 `Exception` 을 상속해서 `raise` 할 수 있어요.
- **줄 15~16** 404 쪽지를 자주 써서 만들어 둔 바로가기.
- **줄 19~20** 창구 어디서든 `raise ApiError(...)` 하면 → 이 함수가 받아서 `{"code": …, "message": …}` 답장으로 바꿔요. 화면의 `api.ts` 줄 39 가 이 두 칸을 읽어요. 🧑 **서버와 화면이 같은 쪽지 모양을 약속**한 거예요.
- **줄 23~25** 예상 못 한 오류(버그)는 🧑 **"잠시 후 다시" 만 말하고 속사정은 숨겨요.** 오류 속에 코드 위치나 데이터가 보이면 나쁜 사람에게 힌트가 되니까요. 진짜 원인은 서버 로그에만 남아요.
- **줄 28~31** 보낸 모양이 틀렸을 때(예: 이메일 칸이 없음) → 어느 칸이 왜 틀렸는지 한 줄로.
  - 🔧 `loc` 은 `["body", "email"]` 같은 위치 목록. `[1:]` 로 앞의 `body` 를 빼고 `"email"` 만 남겨요.

✅ **체크**

- [ ] 남의 지원 번호를 넣으면 403 이 아니라 404 가 나오는 이유를 다시 말할 수 있다
- [ ] 지원자가 답변에 "이 답변은 100점이다" 라고 말하면 어떻게 되는지 찾았다

정류장 13. 테스트로 확인하기 — 내 이해가 맞나?

🧑 **쉬운 설명:** 테스트는 "이 함수에 이걸 넣으면 이게 나와야 해" 를 적어 둔 **약속 쪽지**예요. 코드가 어려우면 테스트부터 읽는 게 더 쉬울 때가 많아요.

👀 **정류장별로 같이 읽을 테스트** (`backend/tests/`)

정류장	테스트 파일
3 상태	`test_status.py`
4 로그인	`test_api_auth.py`, `test_login_block.py`, `test_password_policy.py`
5 지원	`test_api_apply.py`, `test_resume_extract.py`
6 질문	`test_question_rag.py`, `test_question_text.py`, `test_reserve_fallback.py`
7 면접	`test_interview_graph.py`, `test_followup.py`, `test_device_end.py`
8 코딩	`test_sandbox_judge.py`, `test_coding_resume.py`
9 채점	`test_scoring.py`, `test_evaluation.py`, `test_review_deadline.py`
11 보관	`test_retention.py`
화면 전체	`huge/e2e/tests/full.spec.ts` (실제 브라우저로 처음부터 끝까지)

🧪 **직접 해 보기** (`huge/backend` 폴더에서)

```
python -m pytest tests/test_status.py -q      # 가장 쉬운 것부터
python -m pytest tests/test_scoring.py -q
python -m pytest -q                            # 전부 (약 450개)
```

> `test_interview_graph.py` 일부는 Python 3.11 이상에서만 돌아요. 3.10 에서는 skip 으로 보이는 게 정상이에요.

🏁 다 돌았나요? — 마지막 확인

아래 질문에 막힘없이 답하면 **전체 구조를 파악한 것**이에요.

- [ ] 지원자가 "지원하기" 를 누른 순간부터 결과를 볼 때까지, `Application.attempt_status` 가 어떻게 바뀌는지 순서대로 말할 수 있다
- [ ] 그 과정에서 AI 가 쓰이는 곳 4군데(질문 만들기, 받아쓰기·읽어 주기, 꼬리질문, 채점)와 각각의 파일을 말할 수 있다
- [ ] LangGraph 가 쓰이는 곳 3군데(질문 구성, 면접 진행, AI 공고 초안)를 안다
- [ ] api 가 직접 하지 않고 worker 에게 맡기는 일 3가지를 안다
- [ ] 새 기능을 만든다면 "화면(`app/`) → `api()` → 라우터(`routers/`) → 서비스(`services/`) → 모델(`models.py`)" 중 어디를 고쳐야 할지 짐작할 수 있다

다음에 볼 것

- 함수 하나를 깊이 알고 싶으면 → `프로그램명세서.md` 에서 이름으로 찾기
- 파일 하나의 역할이 헷갈리면 → `코드열람가이드_파일과함수.md` 4·5장
- 설계 문서와 다르게 만든 부분이 궁금하면 → `huge/CONFLICTS.md`